Urban Magnetic Navigation in West Coast Park Singapore: Utilising iPhone Magnetometer Data Across Simulated GNSS-Denied Scenarios

Lim Jia Wei

July 28, 2026

Abstract

Global Navigation Satellite Systems (GNSS) are foundational to modern infrastructure, underpinning telecommunications, global transportation, power grids, and financial networks. Without GNSS, spatial coordination and long-range transport face severe operational hurdles. However, satellite navigation remains vulnerable: dense urban infrastructure obstructs line-of-sight signals, while electronic warfare introduces severe risks of jamming and spoofing. To ensure navigation resilience during GNSS outages, recent advances in quantum sensing present a compelling alternative: magnetic based navigation (MagNav). Contextualised within Singapore's dense high-rise and subterranean environment, this report investigates state estimation via Kalman filtering by leveraging both Earth's geomagnetic field and localised, man-made urban magnetic anomalies, offering a robust navigation framework for autonomous and inertial systems operating in GNSS-denied environments.

Contents

1 Introduction **4**

2 Kalman Filtering **4**
2.1 Unscented Kalman Filter 5
2.1.1 Unscented Transform 6
2.1.2 UKF Algorithm 6

3 Sensor Platform and Data Acquisition **7**

4 Navigation System Modelling **9**
4.1 Coordinate Frame Transformations 9
4.2 State-Space Formulation and Process Model 10

5 Magnetometer Fingerprinting **10**
5.1 Data Acquisition Platform Selection 10
5.2 B Field Fingerprint Map Generation 11
5.3 Magnetic Fingerprint Map Reconstruction via Gaussian Process Regression (GPR) 11
5.4 Offline Hyperparameter Optimisation 12
5.5 Real-Time Inference (Predictive Equations) 12
5.6 Discretised Block-Matrix Assembly and Memory-Optimised Inference 13

6 Multi-Sensor Measurement Observation Models **13**

7 Trajectory Alignment and Repeatability Analysis **14**
7.1 1D Spatial Path Linear Interpolation 14
7.2 Zero-Mean Normalised Cross-Correlation (ZNCC) 15
7.3 Point-by-Point Sliding Window Matching 15
7.4 Pairwise Correlation Matrix Assembly 15

8 Results **16**
8.1 Experimental Setup and Testbed Mapping 16
8.2 1D Linear Corridor Repeatability Analysis 17
8.2.1 "Above" Corridor Telemetry and Correlation 17
8.2.2 "Below" Corridor Telemetry and Correlation 17
8.3 2D Spatial Lawn Sweeping Grid Survey 17
8.4 Combined TMI Maps and Reference Map Generation 19
8.5 ZNCC Performance Under Varied Reference Configurations 19

9 Discussion **25**
9.1 Above Path Performance and Anomaly Analysis 25
9.2 Below Path Performance 34
9.3 Implications for Kalman Filtering 34
9.4 Spatial Uniqueness and Cross-Path Disambiguation 34
9.5 Permutation Analysis: Reference Aggregation and Cross-Correlation Edge Cases 34
9.5.1 Positive Reference Behavior ($7:1$ Averaging and Expected Disambiguation) 34
9.5.2 Anomalies, Signal Quality Sensitivity, and Edge Failures 35
9.6 Windowing Constraints and Data Requirements 35
9.7 2D Spatial Field Analysis and Discussion 35
9.7.1 Environmental Noise vs. Sensor Sensitivity 35
9.7.2 Spatial Anomalies and Infrastructure Coupling 35
9.7.3 Methodological Limitations and Dataset Utility 36

10 Future Work **36**
10.1 Expanded 2D Field Surveys and Spatial Cross-Correlation . . . 36
10.2 Gaussian Process Model Validation and Sensitivity Bounds . . . 36
10.3 Adaptive State Estimation and Filter Tuning . . . 36
10.4 Sensor Calibration and Advanced Preprocessing . . . 37

11 Conclusion **37**

12 Appendix A - Quaternions and DCM Derivation **38**
12.1 Quaternions . . . 38
12.2 Quaternion Operations . . . 39
12.3 Direction Cosine Matrix (Direction Cosine Matrix) Derivation . . . 40

13 Appendix B - Gaussian Process Regression (GPR) **42**
13.1 Motivation . . . 42
13.1.1 Least Squares Regression . . . 42
13.2 Foundational Gaussian Process Theory . . . 43
13.2.1 Variance and Covariance . . . 43
13.2.2 Multivariate Gaussian Distributions . . . 44
13.2.3 From Multivariate Gaussians to Gaussian Processes . . . 45
13.3 Real-Time Field Inference at an Arbitrary Point . . . 47
13.4 Kernel Selection . . . 48
13.5 Incorporating Homoscedastic Measurement Noise . . . 50
13.6 Mathematical Deconstruction of Offline Hyperparameter Optimisation . . . 50
13.7 Numerical Block Interleaving and Vectorised Matrix Contractions . . . 51
13.7.1 Block Interleave Index Gymnastics . . . 51
13.7.2 Spatial Voxel Thinning vs. High-Density Evaluation Grid . . . 51

1 Introduction

Global Navigation Satellite Systems (GNSS) serve as the primary foundation for modern positioning, navigation, and timing (PNT) across maritime, aerial, and ground based platforms. However, reliable GNSS performance requires unimpeded line-of-sight (LOS) connectivity to orbiting satellites. In GNSS denied or signal degraded environments such as regions subject to electronic warfare (EW) and jamming, or dense urban canyons affected by electromagnetic interference and multipath shadowing, standalone GNSS becomes insufficient. Consequently, alternative, unjammable positioning technologies are essential for resilient navigation.

Magnetic anomaly navigation (MAGNAV) offers a passive, self contained alternative. By matching real time magnetic field measurements against a calibrated spatial fingerprint, a vehicle or user can infer their position without emitting external signals. Existing research spans both natural Earth anomalies (geomagnetic field) [1, 2] and artificial man made signatures prevalent in urban infrastructure [3, 4, 5].

A primary bottleneck in deploying MAGNAV lies in map availability. Global geomagnetic reference models, such as the World Magnetic Model (WMM) and its high resolution variant (WMMHR), lack the spatial resolution necessary for localised terrestrial positioning. [6] With the baseline WMM resolving features down to approximately $3,330$ km ($n = 12$) and WMMHR extending crustal field resolution to roughly 300 km ($n = 133$) at the equator, these models capture continental-scale core and crustal dynamics rather than fine-grained local field variations. In dense urban environments like Singapore, micro-spatial magnetic anomalies, induced by reinforced concrete, subterranean pipelines, and industrial grid infrastructure, vary on meter or sub-meter scales. As such, localised navigation cannot rely on existing global reference maps and instead requires high-density empirically mapped fingerprints.

While recent breakthroughs in quantum magnetometry, such as Q-CTRL's Ironstone Opal system, demonstrate sub-nanotesla ($<$ nT) sensitivity and field validated quantum assisted positioning [7, 8], widespread commercial and operational deployment remains constrained by hardware availability, cost, and payload considerations. This work investigates whether commercial off-the-shelf (COTS) sensors, specifically those integrated into everyday consumer smartphones, possess sufficient sensitivity, consistency, and spatial repeatability for localised terrestrial magnetic navigation in the absence of a high resolution reference map.

We evaluate this approach in West Coast Park, a regional park in Singapore running parallel to the West Coast Highway and Pasir Panjang Terminal. This site provides a representative testbed, offering a mixed environment of natural terrain, adjacent heavy industrial infrastructure, and transit corridors. Raw inertial and magnetic telemetry were logged using standard mobile interfaces [9] to establish an empirical baseline of magnetic repeatability across repeated trial paths.

Finally, while this report focuses on spatial field mapping and repeatability analysis, magnetometers inherently suffer from noise and temporal drift. [10] Resolving continuous trajectories requires fusing magnetic anomaly observations with Dead Reckoning (DR) or Inertial Navigation Systems (INS). To pave the way for future implementation, this work outlines the dataset structure and preprocessing pipeline required to deploy recursive state estimators, such as Extended or Unscented Kalman Filters [11], and spatial interpolation algorithms like Gaussian Process Regression (GPR) [12] in subsequent research.

2 Kalman Filtering

Much of the content written here was taken from Roger R.Labbe's textbook Kalman and Bayesian Filtering in Python, which provides an intuitive and gentle introduction to Kalman Filtering. At its heart, the Kalman Filter is nothing more than a recursive algorithm that estimates the state of a dynamic system from a series of noisy measurements. [11] Every series of measurements is used to update the estimate of the system's state, which can be anything from position and velocity to more complex variables depending on the application. The basic working principle can be described diagrammatically as follows:

Figure 1 illustrates the concept of residuals in the context of Kalman Filtering. The residual is the difference between the predicted measurement and the actual measurement, which is used to update the state estimate. The Kalman Filter uses this residual to adjust the state estimate and reduce the uncertainty in the estimation process. The previous state of the system is called the posterior while the predicted state is called the prior. The Kalman Filter iteratively updates the state estimate by combining the prior with

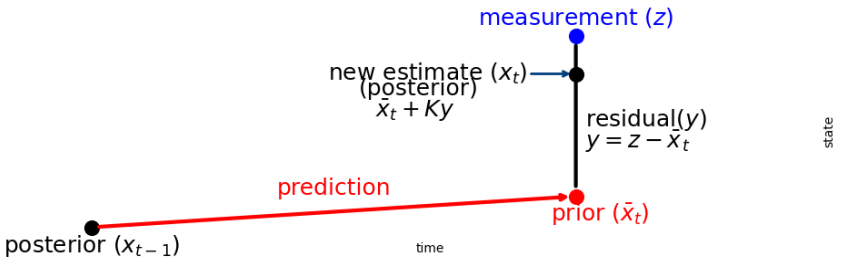


Figure 1: Residual Diagram for Kalman Filter [11]

the new measurement, weighted by the Kalman Gain, which determines how much trust to place in the new measurement versus the prior estimate.

For this report, the state of the dynamic system for navigation comprises of position (x,y,z), velocity and heading, so as to calculate the dead reckoning of the system. The Kalman Filter is particularly well-suited for this task because it can optimally combine information from multiple sources (e.g., inertial sensors, magnetometers) while accounting for uncertainties in both the system dynamics and the noise of the sensors.

The Kalman Filter operates in two main steps, **prediction** followed by **update**. The prediction step uses the current state estimate and a model of the system's dynamics to predict the next state. This involves applying the system's transition model to the current state estimate, which can be represented mathematically as:

$$\bar{x} = Fx + Bu \tag{1}$$

$$\bar{P} = FPF^T + Q \tag{2}$$

where $\bar{x}$ is the predicted state, F is the state transition matrix, x is the current state estimate, B is the control input matrix, and u is the control input, P is the current state estimate error covariance, and Q is the process noise covariance.

The update step incorporates new measurements to refine the state estimate. This involves calculating the Kalman Gain, which determines how much weight to give to the new measurements versus the predicted state. The update equations can be expressed as:

$$y = z - H\bar{x} \tag{3}$$

$$K = \bar{P}H^T(H\bar{P}H^T + R)^{-1} = \bar{P}H^T S^{-1} \tag{4}$$

$$x = \bar{x} + Ky \tag{5}$$

$$P = (I - KH)\bar{P} \tag{6}$$

where y is the measurement residual, z is the actual measurement, H is the measurement matrix, R is the measurement noise covariance, K is the Kalman Gain, and S is the residual covariance.

2.1 Unscented Kalman Filter

While the standard Kalman Filter assumes normally distributed (gaussian) noise and linearity in the state transition and measurement models, many real-world systems are inherently nonlinear, such as the tracking of vehicles for the purpose of navigation.

There are many variants of Kalman Filters that are designed to handle nonlinearities, one notable example would be the Extended Kalman Filter (EKF). For this report, we choose to implement the Unscented Kalman Filter (UKF) over the EKF. It is noted that the UKF is generally simpler to implement than the EKF and also generally performs equal if not better than the EKF. [11] We will therefore choose to implement the UKF for this report.

The core philosophy of the UKF is that it is easier to approximate a gaussian distribution than to linearise a non-linear system (which is what the EKF does). It does so via implementing the Unscented Transform, which is a deterministic sampling technique that captures the mean and covariance of the state distribution.

2.1.1 Unscented Transform

The Unscented Transform is a three step process.

Step 1. Select Sigma Points. Using a deterministic sampling method (in this report we use the Merwe approach) that picks a small cloud of $2n+1$ points, where n is the dimension of the state space. These points are chosen to capture the mean and covariance of the state distribution. Hence, rather than propagating all sample points through the non-linear system, the UKF only propagates these specific Sigma Points.

The Merwe Scaled Sigma Point approach requires three parameters, α, β, and κ, which determines how the Sigma Points are chosen deterministically. α determines the spread of the sigma points around the mean. It is usually set to a small positive value (e.g 10^{-3} to 1). β incorporates prior knowledge about the distribution of the state (for Gaussian distributions, $\beta = 2$ is optimal). κ is a secondary scaling parameter that can be used to fine-tune the distribution of the sigma points. It is usually set to 0 or $3 - n$.

Step 2. Propagate Sigma Points. Each of the sigma points is then propagated through the non-linear system dynamics and measurement models to obtain a new set of transformed sigma points. Unlike the EKF which propagates the Jacobian of the system, the UKF propagates the actual sigma points through the non-linear functions, which allows it to capture the true mean and covariance of the transformed distribution.

Step 3. Recombine to form the new gaussian. The UKF takes the weighted average of these transformed points to calculate the new mean, and a weighted covariance to calculate the new uncertainty.

2.1.2 UKF Algorithm

The UKF algorithm requires additional parameters to augment the basic Kalman Filter. First, the implementation of the Merwe Scaled Sigma Point approach:

$$\mathcal{X} = \text{MerweScaledSigmaPoints}(x, P, \alpha, \beta, \kappa) \tag{7}$$

and the associated weights of the sigma points for calculating the mean and covariance:

$$\omega^m, \omega^c = \text{ComputeMerweWeights}(n, \alpha, \beta, \kappa) \tag{8}$$

Followed by the modified **prediction** step:

$$\mathcal{Y} = f(\mathcal{X}) \tag{9}$$

$$\bar{x} = \sum_{i=0}^{2n} \omega_i^m \mathcal{Y}_i \tag{10}$$

$$\bar{P} = \sum_{i=0}^{2n} \omega_i^c (\mathcal{Y}_i - \bar{x})(\mathcal{Y}_i - \bar{x})^T + Q \tag{11}$$

Here we see that the sigma points are propagated through the non-linear function f to obtain $\mathcal{Y}$, which is then used to calculate the predicted mean $\bar{x}$ and covariance $\bar{P}$ according to the weights ω^m and ω^c for the weights of the mean and covariance respectively.

And finally the modified **update** step:

$$\mathcal{Z} = h(\mathcal{Y}) \tag{12}$$

$$\mu_z = \sum_{i=0}^{2n} \omega_i^m \mathcal{Z}_i \tag{13}$$

$$P_z = \sum_{i=0}^{2n} \omega_i^c (\mathcal{Z}_i - \mu_z)(\mathcal{Z}_i - \mu_z)^T + R \tag{14}$$

$$y = z - \mu_z \tag{15}$$

$$K = \left[\sum_{i=0}^{2n} \omega_i^c (\mathcal{Y}_i - \bar{x})(\mathcal{Z}_i - \mu_z)^T \right] P_z^{-1} \tag{16}$$

$$x = \bar{x} + Ky \tag{17}$$

$$P = \bar{P} - KP_zK^T \tag{18}$$

the predicted sigma points $\mathcal{Y}$ are transformed through the measurement function h to obtain $\mathcal{Z}$ which is the measurements in the measurement space. The mean μ_z and covariance P_z of the predicted measurement are calculated using the weights ω^m and ω^c. The measurement residual y is calculated as the difference between the actual measurement z and the predicted measurement mean μ_z. The Kalman Gain K is calculated using the cross-covariance between the state space $\mathcal{Y}_i - \bar{x}$ and measurement space $\mathcal{Z}_i - \mu_z$. Finally, the state estimate and covariance are updated accordingly.

3 Sensor Platform and Data Acquisition

To validate the performance of the state estimation framework, real-world kinematic and telemetry data were captured using an iPhone via `Sensor Logger` mobile application available on the App Store. [9] Table 1 outlines the complete taxonomy of the extracted sensor modalities, their structural vector formats, and their specific operational roles within the tracking simulation.

Table 1: Taxonomy of Extracted iPhone Sensor Log Data Fields

Sensor Modality	Log Field / Parameters	Data Format	Simulation Role / Application
Accelerometer	`accelerometer xyz`	Vector $\mathbb{R}^3$ (m/s^2)	Primary control input (u_k) driving high-frequency linear dead reckoning velocity integration.
Gravity Vector	`gravity xyz`	Vector $\mathbb{R}^3$ (m/s^2)	Onboard pre-isolated gravity vector relative to the device axes; eliminates the need for manual gravity separation algorithms.
Gyroscope	`gyroscope xyz`	Vector $\mathbb{R}^3$ (rad/s)	Primary control input (u_k) tracking body-frame angular rates to propagate orientation states.
Magnetometer	`magnetometer xyz`	Vector $\mathbb{R}^3$ (μT)	Secondary modality for localisation profiling and anomaly-based fingerprinting mapping.
Compass	`magnetic bearing`	Scalar $\mathbb{R}^1$ (Degrees)	Orientation initialization reference.
Internal Orientation	`qw, qx, qy, qz`	Quaternion $\mathbb{R}^4$ (Dimensionless)	Core attitude parameter mapping the structural coordinate transformation matrix.
Internal Attitude	`roll, pitch, yaw`	Euler Angles $\mathbb{R}^3$ (Radians)	Secondary internal structural logs.
GNSS Coordinates	`latitude, longitude`	Geodetic Coords	Low-frequency absolute horizontal position update (z_k) to bound integration drift.
GNSS Accuracy	`horizontalAccuracy, verticalAccuracy`	Scalar $\mathbb{R}^1$ (Meters)	The radius of uncertainty, approximately the unit standard deviation around the reported position.
GNSS Altitude	`altitude, altitude AMSL`	Elevation (Meters)	Vertical tracking coordinates referenced against the WGS84 ellipsoid and Mean Sea Level.
GNSS Kinematics	`speed, speedAccuracy`	Magnitude (m/s)	Direct velocity magnitude observation inputs.
GNSS Kinematics	`bearing, bearingAccuracy`	Angle (Degrees)	Direct velocity direction heading observations.

4 Navigation System Modelling

To evaluate the telemetry data extracted from the smartphone, a structured navigation model must be established to map the raw sensor readings into the coherent state estimation framework defined above.

4.1 Coordinate Frame Transformations

The first step is to define a consistent coordinate frame for the navigation system. We define the two primary coordinate frames as follows as shown in Figure 2:

1. **Navigation Frame (n-frame):** A global Cartesian coordinate grid fixed to the Earth's surface, oriented along a North-East-Down (NED) alignment. This serves as the global frame for tracking position and velocity.
2. **Body Frame (b-frame):** A local moving coordinate frame aligned with the physical chassis of the iPhone IMU, where the sensor registers raw linear forces and angular rates.

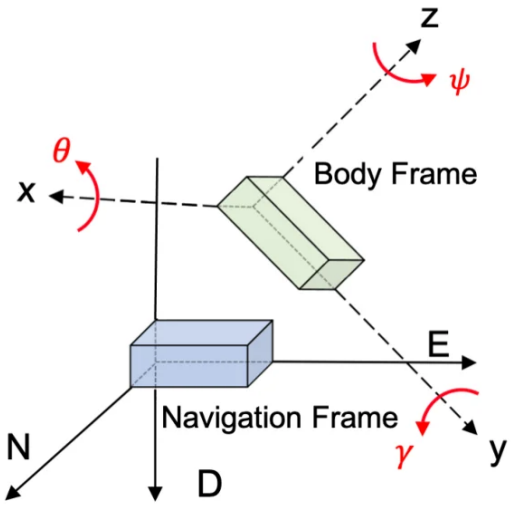


Figure 2: Transformation between navigation frame (n-frame) and the body frame (b-frame). [13]

In order to plot standardised magnetometer data, we need to know how to rotate the b-frame with reference to the n-frame, and perform the adjusted calculations given the b-frame data. We do so using quaternions $q = [q_w, q_x, q_y, q_z]^T$ (rotation operators that describe how to rotate the b-frame with respect to the n-frame) provided by Internal Orientation are utilised at each discrete time step k to construct a time-varying Direction Cosine Matrix (DCM), denoted as $R_b^n(q_k) \in \mathbb{R}^{3\times3}$. This matrix maps vectors directly from the phone's physical body frame into the global navigation frame, given how much the b-frame has rotated against the n-frame:

$$R_b^n(q_k) = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_wq_z) & 2(q_xq_z + q_wq_y) \\ 2(q_xq_y + q_wq_z) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_wq_x) \\ 2(q_xq_z - q_wq_y) & 2(q_yq_z + q_wq_x) & 1-2(q_x^2+q_y^2) \end{bmatrix} \tag{19}$$

We discuss more on the meaning behind quaternions and the derivation of the DCM in Section. 12. Notice that if we are being pedantic, the n-frame actually rotates if we cover significant distances as the earth itself is a sphere. For simplicity and the practical scale of this set up, we will not consider the rotation of the global n-frame and assume it does not rotate.

4.2 State-Space Formulation and Process Model

The movement of the device in the navigation frame is sufficiently modelled in three dimensions for the usual x, y and z axes respectively. As such, the state-space $\bar{x}_k$ formulation of the filter is defined as follows, 3D position and 3D velocity:

$$x_k = \begin{bmatrix} p_N & p_E & p_D & v_N & v_E & v_D \end{bmatrix}^T \tag{20}$$

The sensor's accelerometer data is directly utilised to construct the control input $u_k \in \mathbb{R}^3$. Since the accelerometer readings $a_{b,k}$ already report linear dynamic acceleration in $\mathrm{m/s}^2$ with gravity isolated, $a_{b,k}$ directly represents the true body-frame acceleration of the device.

The net body acceleration is subsequently projected into the global navigation grid via the Direction Cosine Matrix (DCM) rotation matrix $R_b^n(q_k)$ for a particular quaternion value of q_k at time step k, yielding the control input vector u_k for the state transition model:

$$u_k = a_{n,k} = R_b^n(q_k) a_{b,k} \tag{21}$$

Lastly, by applying Newtonian kinematic transitions over a fixed time step Δt, the continuous-time physics equations are mapped directly into the linear discrete-time system state equation structure previously outlined in Eq. (1). For this specific navigation state-space instantiation, the structural transition matrix $F \in \mathbb{R}^{6\times 6}$ and the control mapping matrix $B \in \mathbb{R}^{6\times 3}$ are parameterised explicitly as:

$$F = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{2}\Delta t^2 & 0 & 0 \\ 0 & \frac{1}{2}\Delta t^2 & 0 \\ 0 & 0 & \frac{1}{2}\Delta t^2 \\ \Delta t & 0 & 0 \\ 0 & \Delta t & 0 \\ 0 & 0 & \Delta t \end{bmatrix} \tag{22}$$

The process noise covariance matrix $Q \in \mathbb{R}^{6\times 6}$, injected during the covariance propagation phase in Eq. (2), models the structural uncertainty and stochastic velocity random walk accumulated during uncorrected dead reckoning blocks (for example during GNSS-denied segments where the filter relies solely on the control input u_k to propagate the state). The specific values within Q need to be tuned empirically based on the expected noise characteristics of the accelerometer and gyroscope data, as well as the anticipated dynamics of the device's movement.

5 Magnetometer Fingerprinting

This implementation requires two distinct phases. Firstly, an offline spatial field reconstruction phase that constructs a continuous space magnetic map across the target grid, and secondly, the real time processing and fingerprinting of live magnetometer data against this background map during navigation.

5.1 Data Acquisition Platform Selection

To establish the baseline mapping platform, two potential commercial off-the-shelf (COTS) candidates were considered: a standard consumer smartphone (iPhone) and a micro-drone platform (DJI Neo).

While the DJI Neo offers high spatial mobility, detailed technical documentation and low-level API access regarding its internal magnetometer specifications and telemetry broadcast pipeline were not available at the time of writing. Furthermore, we suspect that small aerial platforms frequently suffer from severe localised electromagnetic interference (EMI) generated by high-current brushless motor windings in close proximity to onboard sensors.

In light of DJI Neo's complexities, the iPhone was selected as the primary instrumentation platform for both the offline mapping and real time processing phase. Access to raw sensor data through standardised logging interfaces enables high rate sampling with transparent software calibration, providing a consistent and low-noise observation baseline to train the continuous spatial Gaussian Process model.

5.2 B Field Fingerprint Map Generation

As with the accelerometer data, the magnetometer data produced by the mobile sensors are outputted in the b-frame. We use the DCM, R_b^n, with the quaternion data from the sensor data to transform the B field measurements in the b-frame to the n-frame:

$$\mathbf{B}_n = R_b^n \mathbf{B}_b \tag{23}$$

However, errors in attitude tracking, for example heading drift and high-frequency vibrations from propulsion systems, propagate directly into the rotated vector components, yielding a corrupted navigation-frame vector $\mathbf{B}_{n,k} + \delta\mathbf{B}_{n,k}$.

To mitigate this, we augment our fingerprint vector with the magnitude of the B field vector, also known as the Total Magnetic Intensity (TMI) scalar component, computed via the L_2-norm of the sensor measurements. Because rotations are invariant of the L_2-norm space and the DCM, $R_b^n(q_k)$, is orthogonal, the scalar field magnitude $F_k \in \mathbb{R}$ remains identical in both reference frames. This provides another measurement quantity for the Kalman Filter to consider, as we know that the TMI value should be conserved. If the TMI value is not conserved, then there must have been B field leakage, and hence the Kalman Filter will tend to trust the B field fingerprinting less.

$$F_{\text{sensor},k} = \|\mathbf{B}_{n,k}\|_2 = \sqrt{B_N^2 + B_E^2 + B_D^2} = \sqrt{B_x^2 + B_y^2 + B_z^2} = \|\mathbf{B}_{b,k}\|_2 \tag{24}$$

As such, our magnetic fingerprinting dataset $\mathcal{D}$ tracks both the 3D directional field vector components and the absolute TMI intensity at every discrete coordinate location $\mathbf{x}_i = [x_i, y_i, z_i]^T \in \mathbb{R}^3$:

$$\mathcal{D} = \{(\mathbf{x}_i,\, B_{N,i},\, B_{E,i},\, B_{D,i},\, F_i)\}_{i=1}^N \tag{25}$$

5.3 Magnetic Fingerprint Map Reconstruction via Gaussian Process Regression (GPR)

To reconstruct the magnetic field map, Gaussian Process Regression (GPR) serves as the primary method in this work, following established validation methodologies in quantum magnetometry. [1] By framing map reconstruction through GPR, discrete spatial sensor measurements in $\mathcal{D}$ are modeled as a smooth, continuous representation $f(\mathbf{x})$, enabling probabilistic interpolation and extrapolation at unobserved coordinates.

$$f(\mathbf{x}) \sim \mathcal{GP}\left(m(\mathbf{x}), \mathbf{K}_{\text{field}}(\mathbf{x}, \mathbf{x}')\right) \tag{26}$$

where $m(\mathbf{x})$ is the prior mean function, and $\mathbf{K}_{\text{field}}(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^{3\times 3}$ is the matrix-valued kernel which enforces constraints $\nabla \times \mathbf{B} = \mathbf{0}$ and $\nabla \cdot \mathbf{B} = 0$:

$$\mathbf{K}_{\text{field}}(\mathbf{x}, \mathbf{x}') = \nabla \nabla'^T K(\mathbf{x}, \mathbf{x}') \tag{27}$$

Choosing the Squared Exponential covariance function for the underlying latent scalar potential base kernel:

$$K(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{|\mathbf{x} - \mathbf{x}'|^2}{2\ell^2}\right) \tag{28}$$

Evaluating the Hessian matrix yields the explicit analytical parameterisation:

$$\mathbf{K}_{\text{field}}(\mathbf{x}, \mathbf{x}') = \frac{\sigma_f^2}{\ell^2} \exp\left(-\frac{|\mathbf{x} - \mathbf{x}'|^2}{2\ell^2}\right) \left[\mathbf{I}_3 - \frac{(\mathbf{x} - \mathbf{x}')(\mathbf{x} - \mathbf{x}')^T}{\ell^2}\right] \tag{29}$$

For a surveyed spatial training dataset $\mathcal{D}$ containing N points, the full multi-variable covariance matrix $\mathbf{K}_y \in \mathbb{R}^{3N\times 3N}$ accounts for homoscedastic sensor measurement noise variance σ_n^2:

$$\mathbf{K}_y = \mathbf{K}(X, X') + \sigma_n^2 \mathbf{I}_{3N} \tag{30}$$

5.4 Offline Hyperparameter Optimisation

To accurately model both the directional components and the absolute intensity of the magnetic field, our pipeline splits the Gaussian Process into two parallel tracks: Track A for the 3D Vector field and Track B for the 1D Scalar Total Magnetic Intensity (TMI) field.

The optimal hyperparameter vectors for both tracks are resolved offline by maximising the Marginal Log-Likelihood (MLL) objective function using the L-BFGS-B optimisation algorithm. To ensure numerical stability and physical realism, the optimiser is constrained by the following bounds: characteristic length-scale $\ell \in [0.1, 5.0]$ m, signal variance $\sigma_f \in [0.1, 10.0]\,\mu$T, and a measurement noise floor $\sigma_n \in [0.01, 1.0]\,\mu$T.

Track A: Vector Field Optimisation. We optimise the parameters $\boldsymbol{\theta}^*_{\text{vector}} = \{\ell_v, \sigma^2_{f,v}, \sigma^2_{n,v}\}$ over the $3N \times 3N$ curl-free covariance matrix $\mathbf{K}_{y,v}$:

$$\boldsymbol{\theta}^*_{\text{vector}} = \arg\max_{\boldsymbol{\theta}} \left(-\frac{1}{2}\mathbf{y}_v^T \mathbf{K}_{y,v}^{-1} \mathbf{y}_v - \frac{1}{2}\log\det\left(\mathbf{K}_{y,v}\right) - \frac{3N}{2}\log(2\pi) \right) \tag{31}$$

Track B: Scalar TMI Optimisation. Simultaneously, we optimise the parameters $\boldsymbol{\theta}^*_{\text{scalar}} = \{\ell_s, \sigma^2_{f,s}, \sigma^2_{n,s}\}$ over a standard $N \times N$ Squared Exponential (SE) matrix $\mathbf{K}_{y,s}$:

$$\boldsymbol{\theta}^*_{\text{scalar}} = \arg\max_{\boldsymbol{\theta}} \left(-\frac{1}{2}\mathbf{y}_s^T \mathbf{K}_{y,s}^{-1} \mathbf{y}_s - \frac{1}{2}\log\det\left(\mathbf{K}_{y,s}\right) - \frac{N}{2}\log(2\pi) \right) \tag{32}$$

5.5 Real-Time Inference (Predictive Equations)

A standard assumption in Gaussian Process Regression is a zero-mean prior, $m(\mathbf{x}) = \mathbf{0}$, which simplifies the analytical conditioning. However, the true ambient magnetic field is non-zero. To satisfy the zero-mean prior assumption mathematically while preserving the true physical field values, we apply explicit empirical mean-centering to our training data before inference.

We first compute the constant empirical mean vector $\mathbf{B}_0$ and scalar mean F_0 from our spatially thinned dataset of N points:

$$\mathbf{B}_0 = \frac{1}{N}\sum_{i=1}^{N} \mathbf{B}_i, \quad F_0 = \frac{1}{N}\sum_{i=1}^{N} F_i \tag{33}$$

The target vectors $\mathbf{y}_v$ and $\mathbf{y}_s$ fed into the MLL optimiser and inference equations are the mean-centered residuals:

$$\mathbf{y}_v = (\mathbf{B}_{\text{train}} - \mathbf{B}_0)_{\text{flat}}, \quad \mathbf{y}_s = \mathbf{F}_{\text{train}} - F_0 \tag{34}$$

During real-time inference, or when baking our validation grid tensor, the analytical predictive mean $\boldsymbol{\mu}(\mathbf{x}_*)$ and epistemic covariance $\boldsymbol{\sigma}^2(\mathbf{x}_*)$ at an unvisited evaluation coordinate $\mathbf{x}_*$ are evaluated and the empirical means are added back to map the residuals to global coordinates.

For the 3D Vector track:

$$\boldsymbol{\mu}_{\mathbf{B}}(\mathbf{x}_*) = \mathbf{B}_0 + \mathbf{k}_{*,v}^T \mathbf{K}_{y,v}^{-1} \mathbf{y}_v \tag{35}$$

$$\boldsymbol{\sigma}^2_{\mathbf{B}}(\mathbf{x}_*) = \mathbf{K}_{\text{field}}(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_{*,v}^T \mathbf{K}_{y,v}^{-1} \mathbf{k}_{*,v} \tag{36}$$

where $\mathbf{k}_{*,v} \in \mathbb{R}^{3N \times 3}$ is the curl-free cross-covariance mapping.

For the 1D Scalar TMI track:

$$\mu_F(\mathbf{x}_*) = F_0 + \mathbf{k}_{*,s}^T \mathbf{K}_{y,s}^{-1} \mathbf{y}_s \tag{37}$$

$$\sigma^2_F(\mathbf{x}_*) = \sigma^2_{f,s} - \mathbf{k}_{*,s}^T \mathbf{K}_{y,s}^{-1} \mathbf{k}_{*,s} \tag{38}$$

where $\mathbf{k}_{*,s} \in \mathbb{R}^{N \times 1}$ is the standard SE cross-covariance vector.

5.6 Discretised Block-Matrix Assembly and Memory-Optimised Inference

While the analytical formulation in Eq. (**??**) describes the continuous spatial correlation between two arbitrary vectors, processing a thinned dataset $\mathcal{D}$ containing N unique spatial voxels requires mapping this physics onto a discrete numerical computing architecture.

Because each spatial coordinate pairing interacts across a 3×3 matrix operator, compiling the global training matrix $\mathbf{K}_y \in \mathbb{R}^{3N \times 3N}$ requires structured block interleave slicing. For our computational pipeline, we flatten our 3D training observation vector field component-wise into a monolithic $3N \times 1$ column vector:

$$\mathbf{y}_{\text{flat}} = \left[B_{N,1}, B_{E,1}, B_{D,1}, \ldots, B_{N,N}, B_{E,N}, B_{D,N}\right]^T \tag{39}$$

The global matrix structure is assembled by assigning each distinct spatial dimension block step-wise, where the homoscedastic instrumentation noise floor σ_n^2 regularizes the system along the main diagonal:

$$\mathbf{K}_y = \mathbf{K}(\mathbf{X}_{\text{train}}, \mathbf{X}_{\text{train}}) + \sigma_n^2 \mathbf{I}_{3N} \tag{40}$$

At run-time, querying a high-density, three-dimensional validation meshgrid containing N_{grid} evaluation points poses a critical computational bottleneck. Evaluating the predictive variance exactly using traditional matrix algebra requires computing:

$$\mathbf{\Sigma}_* = \mathbf{K}_{\text{field}}(\mathbf{X}_*, \mathbf{X}_*) - \mathbf{K}_* \mathbf{K}_y^{-1} \mathbf{K}_*^T \tag{41}$$

Because $\mathbf{K}_* \in \mathbb{R}^{3N_{\text{grid}} \times 3N}$, a direct computation of the full cross-axis matrix product would scale at a catastrophic memory complexity of $\mathcal{O}((3N_{\text{grid}})^2)$. This would saturate system RAM when baking high-resolution navigation lookup grids.

To overcome this, our pipeline leverages an element-wise reduction identity that isolates only the diagonal elements of the transformed space. By evaluating an intermediate projection tensor via a linear solver interface ($\mathbf{K}_y \mathbf{M} = \mathbf{K}_*^T$), we recover the localised epistemic structural uncertainty bounds $\boldsymbol{\sigma}_{\mathbf{B}}^2(\mathbf{x}_*)$ directly via a vectorised row-sum contraction:

$$\text{diag}(\mathbf{\Sigma}_*) = \sigma_f^2 - \sum_{\text{rows}} \left(\mathbf{K}_* \odot \mathbf{M}^T\right) \tag{42}$$

where $\odot$ denotes the Hadamard element-wise product. This numerical modification reduces memory scaling to a strictly bounded linear footprint, allowing us to successfully bake continuous, high-fidelity vector field lookup snapshots ($\mathcal{T}_{\text{map}}$) stored as discrete uncoupled array tensors.

6 Multi-Sensor Measurement Observation Models

Alongside the acceleration data from the accelerometer unit, the Kalman Filter receives two other types of data. The first is the GNSS data (`GNSS Coordinates, GNSS Accuracy, GNSS Altitude, GNSS Kinematics`). The second is the hybrid B field readings from the magnetometer unit, which are compared directly to the pre-mapped continuous profile tensor $\mathcal{T}_{\text{map}}$ derived via the GPR framework.

As such, our full measurement vector $z_k \in \mathbb{R}^{10}$ at time step k is described as follows:

$$z_k = \begin{bmatrix} p_{N,\text{GNSS}} \\ p_{E,\text{GNSS}} \\ p_{D,\text{GNSS}} \\ v_{N,\text{GNSS}} \\ v_{E,\text{GNSS}} \\ v_{D,\text{GNSS}} \\ B_{N,\text{mag}} \\ B_{E,\text{mag}} \\ B_{D,\text{mag}} \\ F_{\text{sensor}} \end{bmatrix} = \begin{bmatrix} \text{Latitude converted to local meters (North)} \\ \text{Longitude converted to local meters (East)} \\ -\texttt{altitude_AMSL} \text{ (Vertical Position Down)} \\ \texttt{speed} \cdot \cos(\texttt{bearing}) \text{ (Velocity North)} \\ \texttt{speed} \cdot \sin(\texttt{bearing}) \text{ (Velocity East)} \\ \text{Derived rate of altitude change (Velocity Down)} \\ [R_b^n(q_k) \cdot \mathbf{B}_{b,k}]_{\text{Row 1}} \\ [R_b^n(q_k) \cdot \mathbf{B}_{b,k}]_{\text{Row 2}} \\ [R_b^n(q_k) \cdot \mathbf{B}_{b,k}]_{\text{Row 3}} \\ F_{\text{sensor},k} \end{bmatrix} \tag{43}$$

Using zero indexing, rows 0 to 2 consist of what the GNSS thinks the device's location is for the given n-frame axes. Notice that $p_{D,\text{GNSS}}$ is taken to be the negative of the `altitude_AMSL` due to the Down orientation of the NED frame coordinate structure. Rows 3 to 5 consists of what the GNSS thinks the device's velocity is for the given n-frame axes likewise. Rows 6 to 8 contain the transformed B field readings in the n-frame, and Row 9 tracks the frame invariant scalar magnitude value of the B field.

The corresponding $h(x_k)$ measurement transformation assuming all data types are available for the Kalman Filter is described below:

$$h(x_k) = \begin{bmatrix} p_N \\ p_E \\ p_D \\ v_N \\ v_E \\ v_D \\ M_N(p_N, p_E, p_D) \\ M_E(p_N, p_E, p_D) \\ M_D(p_N, p_E, p_D) \\ M_F(p_N, p_E, p_D) \end{bmatrix} \tag{44}$$

With the corresponding measurement noise covariance matrix $R \in \mathbb{R}^{10\times 10}$ structured as a block-diagonal matrix:

$$R_k = \text{diag}\left(\sigma^2_{p_N},\, \sigma^2_{p_E},\, \sigma^2_{p_D},\, \sigma^2_{v_N},\, \sigma^2_{v_E},\, \sigma^2_{v_D},\, \sigma^2_B,\, \sigma^2_B,\, \sigma^2_B,\, \sigma^2_F(\mathbf{x}_k)\right) \tag{45}$$

where the horizontal ($\sigma^2_{p_E}$) and vertical ($\sigma^2_{p_N}$) position variances are mapped from Sensor Logger's accuracy channels, and σ^2_B represents the combined uncertainty of the magnetometer sensor noise and local gradient errors. The parameter σ^2_F tracks the associated lookup error variance of the localised scalar GPR map cell.

In the event of a simulated GNSS blackout, the observation vector dynamically collapses to isolate only the magnetic tracking field components, reducing to a 4-dimensional vector space, $z_{k,\text{denied}} \in \mathbb{R}^4$:

$$z_{k,\text{denied}} = \begin{bmatrix} B_{N,\text{mag}} \\ B_{E,\text{mag}} \\ B_{D,\text{mag}} \\ F_{\text{sensor}} \end{bmatrix}, \quad h_{\text{denied}}(x_k) = \begin{bmatrix} M_N(p_N, p_E, p_D) \\ M_E(p_N, p_E, p_D) \\ M_D(p_N, p_E, p_D) \\ M_F(p_N, p_E, p_D) \end{bmatrix} \tag{46}$$

7 Trajectory Alignment and Repeatability Analysis

Deploying magnetic anomaly navigation requires verifying that localised field variations are spatially repeatable across time and independent of minor variations in movement speed, device orientation, or holding posture. Because mobile telemetry is logged in the moving body frame at discrete time intervals ($\Delta t_{\text{IMU}} \approx 0.01$ s), raw sensor logs sampled across separate passes of the same physical path vary in sample count, attitude, and spatial density.

Using the Direction Cosine Matrix $R^n_b(q_k)$ and L_2-norm defined in Section 4, raw magnetometer observations are rotated into the navigation frame ($\mathbf{B}_n$) and converted to total magnetic intensity (F). To perform cross-correlation analysis across trial runs, these telemetry series are mapped onto a spatially indexed coordinate system, resampled onto a unified spatial reference path, and evaluated using Zero-Mean Normalised Cross-Correlation (ZNCC).

7.1 1D Spatial Path Linear Interpolation

Let a single trial run m consist of K_m time-stamped spatial observations tracking a 1D displacement $s \in [0, L]$ along a mapped path of length L. The rotated field components and scalar intensity yield non-uniformly spaced spatial observation logs:

$$\mathcal{D}_m = \{(s_{m,k},\, B_{N,m,k},\, B_{E,m,k},\, B_{D,m,k},\, F_{m,k})\}_{k=1}^{K_m} \tag{47}$$

To enable direct vector comparisons across separate runs, each scalar channel $y \in \{B_N, B_E, B_D, F\}$ within $\mathcal{D}_m$ is resampled onto a uniform spatial grid $\mathbf{s}^* = [s_0, s_1, \ldots, s_N]^T$ with a fixed spatial sampling interval $\Delta s = 0.1$ m using 1D Piecewise Linear Interpolation:

$$\hat{y}_m(s) = y_{m,k} + \frac{y_{m,k+1} - y_{m,k}}{s_{m,k+1} - s_{m,k}}(s - s_{m,k}) \quad \text{for } s \in [s_{m,k}, s_{m,k+1}] \tag{48}$$

Discretizing $\hat{y}_m(s)$ across the uniform spatial grid $\mathbf{s}^*$ constructs a fixed-length spatial observation vector $\mathbf{x}_m \in \mathbb{R}^N$:

$$\mathbf{x}_m = \begin{bmatrix} \hat{y}_m(s_0), & \hat{y}_m(s_1), & \ldots, & \hat{y}_m(s_N) \end{bmatrix}^T \tag{49}$$

7.2 Zero-Mean Normalised Cross-Correlation (ZNCC)

To evaluate signal similarity independent of global DC magnetic offsets or sensor scale variations, we compute the Zero-Mean Normalised Cross-Correlation (ZNCC) coefficient γ_{ab} between test trajectory $\mathbf{x}_a$ and reference trajectory $\mathbf{x}_b$.

Each spatially interpolated observation vector $\mathbf{x}_m$ is centered by subtracting its empirical spatial mean $\bar{x}_m$:

$$\tilde{\mathbf{x}}_m = \mathbf{x}_m - \bar{x}_m \mathbf{1}_N, \quad \text{where } \bar{x}_m = \frac{1}{N}\sum_{i=1}^{N} [\mathbf{x}_m]_i \tag{50}$$

The ZNCC coefficient $\gamma_{ab} \in [-1, 1]$ between trial runs a and b is evaluated as:

$$\gamma_{ab} = \frac{\tilde{\mathbf{x}}_a^T \tilde{\mathbf{x}}_b}{\|\tilde{\mathbf{x}}_a\|_2 \|\tilde{\mathbf{x}}_b\|_2} = \frac{\sum_{i=1}^{N} \left([\mathbf{x}_a]_i - \bar{x}_a\right)\left([\mathbf{x}_b]_i - \bar{x}_b\right)}{\sqrt{\sum_{i=1}^{N} \left([\mathbf{x}_a]_i - \bar{x}_a\right)^2} \sqrt{\sum_{i=1}^{N} \left([\mathbf{x}_b]_i - \bar{x}_b\right)^2}} \tag{51}$$

A score of $\gamma_{ab} = 1$ denotes perfect spatial profile alignment, $\gamma_{ab} = 0$ indicates uncorrelated signals, and $\gamma_{ab} = -1$ signifies phase inversion.

7.3 Point-by-Point Sliding Window Matching

When matching a live test signal segment against a continuous reference map $\mathbf{x}_{\text{ref}}$, localised alignment is evaluated by sliding a candidate window of length W across the reference signal. For a spatial lag index k, the localised ZNCC score $\gamma(k)$ evaluating a test segment $\mathbf{x}_{\text{test}} \in \mathbb{R}^W$ against sub-window $\mathbf{x}_{\text{ref}}[k : k + W - 1]$ is defined as:

$$\gamma(k) = \frac{\sum_{j=1}^{W} \left([\mathbf{x}_{\text{test}}]_j - \bar{x}_{\text{test}}\right)\left([\mathbf{x}_{\text{ref}}]_{k+j-1} - \bar{x}_{\text{ref},k}\right)}{\sqrt{\sum_{j=1}^{W} \left([\mathbf{x}_{\text{test}}]_j - \bar{x}_{\text{test}}\right)^2} \sqrt{\sum_{j=1}^{W} \left([\mathbf{x}_{\text{ref}}]_{k+j-1} - \bar{x}_{\text{ref},k}\right)^2}} \tag{52}$$

where $\bar{x}_{\text{ref},k} = \frac{1}{W}\sum_{j=1}^{W} [\mathbf{x}_{\text{ref}}]_{k+j-1}$ is the local mean bounded within the sliding window. The estimated spatial coordinate s^* corresponds to the global peak of the correlation profile:

$$s^* = \arg\max_k \left(\gamma(k)\right) \cdot \Delta s \tag{53}$$

7.4 Pairwise Correlation Matrix Assembly

To assess global structural consistency across an ensemble of M trial runs, we assemble an $M \times M$ symmetric Pairwise Correlation Matrix $\mathbf{\Gamma} \in \mathbb{R}^{M \times M}$. Given the matrix of mean-centered spatial vectors $\mathbf{X} = [\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \ldots, \tilde{\mathbf{x}}_M] \in \mathbb{R}^{N \times M}$ and diagonal norm matrix $\mathbf{D} = \text{diag}(\|\tilde{\mathbf{x}}_1\|_2, \ldots, \|\tilde{\mathbf{x}}_M\|_2)$, the complete correlation matrix is computed as:

$$\mathbf{\Gamma} = \mathbf{D}^{-1}\left(\mathbf{X}^T\mathbf{X}\right)\mathbf{D}^{-1} = \begin{bmatrix} 1 & \gamma_{1,2} & \cdots & \gamma_{1,M} \\ \gamma_{2,1} & 1 & \cdots & \gamma_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ \gamma_{M,1} & \gamma_{M,2} & \cdots & 1 \end{bmatrix} \tag{54}$$

High off-diagonal values ($\gamma_{a,b} > 0.85$) confirm that COTS smartphone magnetometers capture highly repeatable micro-spatial anomalies suitable for observation updates in the navigation filter.

8 Results

8.1 Experimental Setup and Testbed Mapping

To evaluate the empirical performance and repeatability of COTS smartphone magnetometers, field telemetry was collected across two distinct survey profiles at West Coast Park, Singapore:

1. **1D Linear Corridors:** Two parallel straight walking paths—designated as the "above" (upper, orange) and "below" (lower, blue) routes—separated by approximately 15 m to 20 m along the park perimeter (Figure 3). Four trial passes (two forward, two backward) were executed per path, yielding eight total 1D benchmark runs.

2. **2D Spatial Area Coverage:** A continuous 2D lawn sweeping grid path conducted on the open field in front of the grandstand to capture high-density spatial field variations across an area footprint.

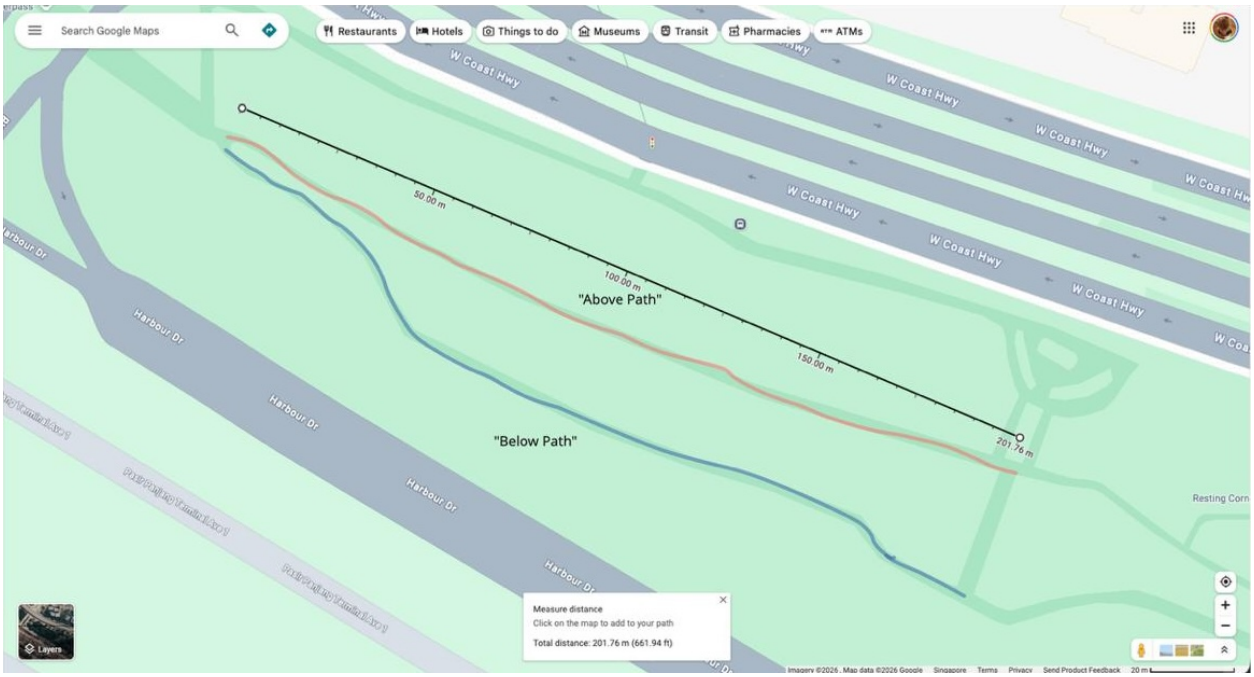


Figure 3: Straight linear paths ("above" in orange, "below" in blue) selected for repeatability and cross-correlation analysis at West Coast Park.

Across all trial runs, an iPhone running the `Sensor Logger` application [9] was held parallel to the ground at waist level ($\approx$ 1 m elevation) against the operator's chest, as shown in Figure 4. Maintaining a fixed holding posture and constant elevation minimizes platform-induced attitude perturbations and ensures that localized ground anomaly observations remain spatially consistent across separate passes. Walking speeds were maintained at a deliberate, uniform slow pace to ensure high spatial sampling density.

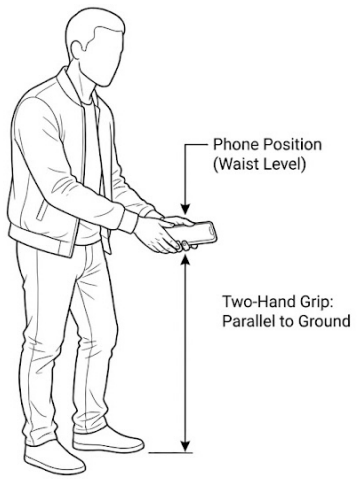


Figure 4: Experimental data collection posture: COTS smartphone held fixed against the operator's waist at approximately 1 m elevation.

Raw telemetry—including IMU accelerations, magnetic field vectors, and internal attitude quaternions—was logged at high frequency ($\Delta t_{\text{IMU}} \approx 0.01$ s). Quaternions were applied via Eq. (19) to transform raw b-frame field measurements into the global North-East-Down (n-frame) system, eliminating device orientation dependencies prior to spatial analysis.

8.2 1D Linear Corridor Repeatability Analysis

To establish a baseline for spatial profile matching, an auto-correlation ZNCC matrix is computed for a reference run against itself. As visualised in Figure 5, correlating a signal against its own spatial trace yields a sharp, ideal diagonal line with unit score ($\gamma = 1.0$), confirming exact coordinate alignment.

8.2.1 "Above" Corridor Telemetry and Correlation

Figure 6 presents the individual rotated 2D magnetic vector components across the four trial passes along the "above" path. To illustrate spatial consistency across separate passes, the rotated trajectories are combined into a 3D vector overlay in Figure 7.

Cross-run ZNCC correlation matrices for the "above" path are presented in Figure 8. The persistent high-intensity diagonal confirms high spatial repeatability across multiple passes along the same physical path.

8.2.2 "Below" Corridor Telemetry and Correlation

An identical processing workflow was applied to the "below" corridor data. The individual directional vector channels are plotted in Figure 9, with their combined 3D spatial overlay shown in Figure 10. Pairwise ZNCC correlation matrices verifying spatial signal agreement along the "below" path are shown in Figure 11.

8.3 2D Spatial Lawn Sweeping Grid Survey

To extend beyond one-dimensional linear profiles, a two-dimensional area mapping survey was conducted on the lawn adjacent to the West Coast Park grandstand. A continuous, high-density lawnmower sweeping pattern was executed across the field during a single 40-minute logging session, covering a cumulative path length of approximately 2 to 3 km.

This dataset provides dense, multi-directional spatial coverage over a continuous planar domain, capturing two-dimensional magnetic field gradients induced by subterranean park infrastructure and local structural anomalies.

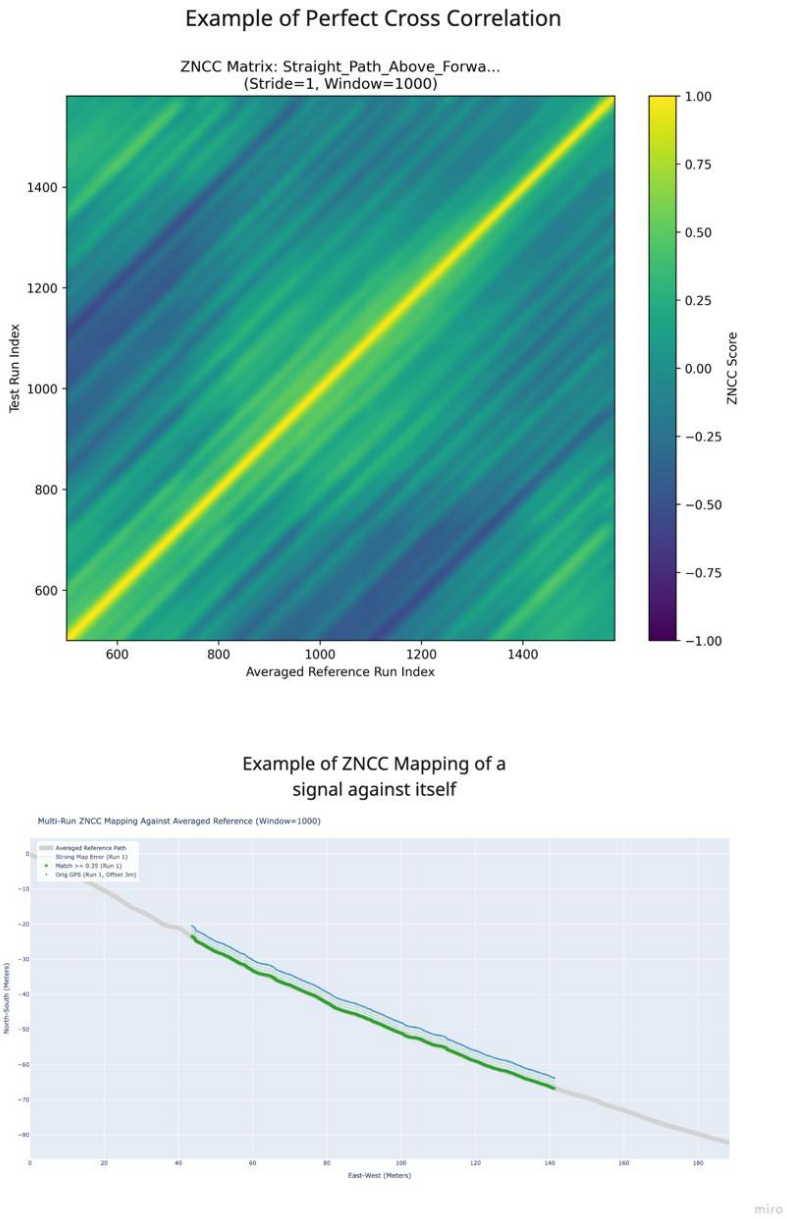


Figure 5: Auto-correlation ZNCC baseline matrix generated by evaluating a single pass against itself, illustrating an ideal coordinate-to-coordinate match along the main diagonal.

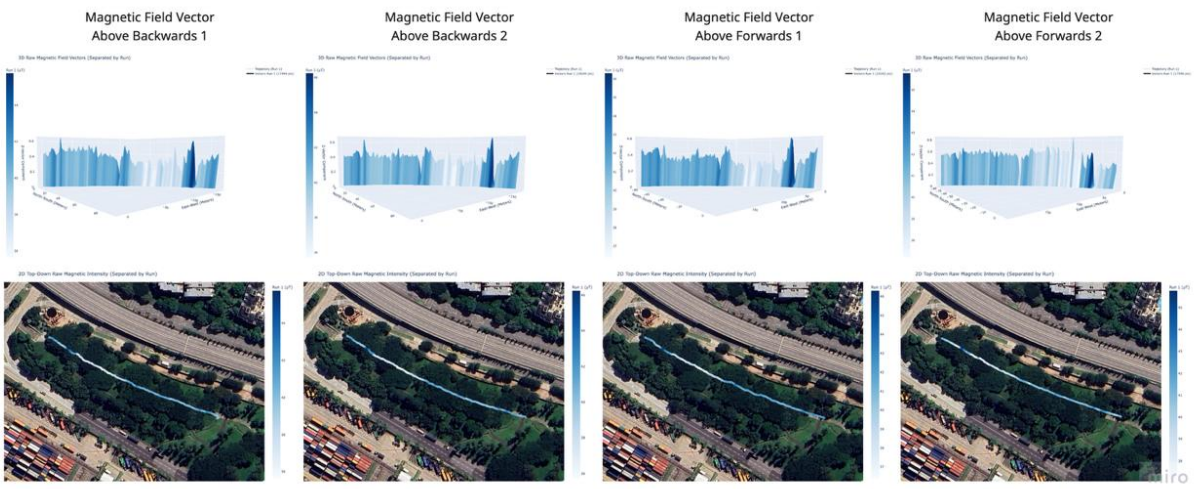


Figure 6: Individual 2D magnetic field vectors (B_N, B_E, B_D) recorded across four passes (two forward, two backward) along the "above" path.

Figure 12 illustrates the reconstructed geodetic trajectory and total magnetic intensity overlaid on top-down satellite imagery. Figure 13 presents the 3D spatial distribution of the vector readings alongside the interpolated scalar field following median subtraction. By removing the spatial median to center the dataset at zero mean, the background geomagnetic field is suppressed, isolating local magnetic anomalies. The resulting high-density sampling serves as the training data for the Maxwell-conforming GPR interpolation kernel detailed in Section 5.

8.4 Combined TMI Maps and Reference Map Generation

To build baseline offline reference maps for matching trials, observations across multiple runs were aggregated into spatially averaged reference profiles. Figure 14 displays the combined Total Magnetic Intensity (TMI) mapped onto local geographic coordinates, and Figure 15 displays the aggregated 3D vector fields across both linear paths.

Spatial averaging across repeated passes attenuates transient, high-frequency sensor noise while isolating static, spatially repeatable magnetic anomalies suitable for reference map construction.

8.5 ZNCC Performance Under Varied Reference Configurations

To evaluate signal-matching sensitivity against reference map quality, individual test runs were cross-correlated against reference pools constructed using three distinct ratios:

- **Full Reference ($7:1$ Ratio):** The reference map is formed by averaging all 7 remaining passes across both paths to evaluate 1 target test run.
- **Half Reference ($3:1$ Ratio):** The reference map is constructed using only the 3 remaining passes sharing the same path and direction category.
- **Cross-Path Reference ($4:1$ Ratio):** The reference map is formed using all 4 passes from one corridor (e.g., "above") to evaluate a test run from the opposing corridor (e.g., "below").

Figures 16 and 17 illustrate the ZNCC signal alignment and correlation matrix for an "above" backward run evaluated against a $3:1$ half-reference pool.

Expanding the reference pool to a $7:1$ full-reference configuration enhances profile stability, as demonstrated in Figures 18 and 19.

Additional evaluation permutations are shown in Figures 20, 21, and 22, highlighting correlation performance across forward passes, backward passes, and cross-path evaluations.

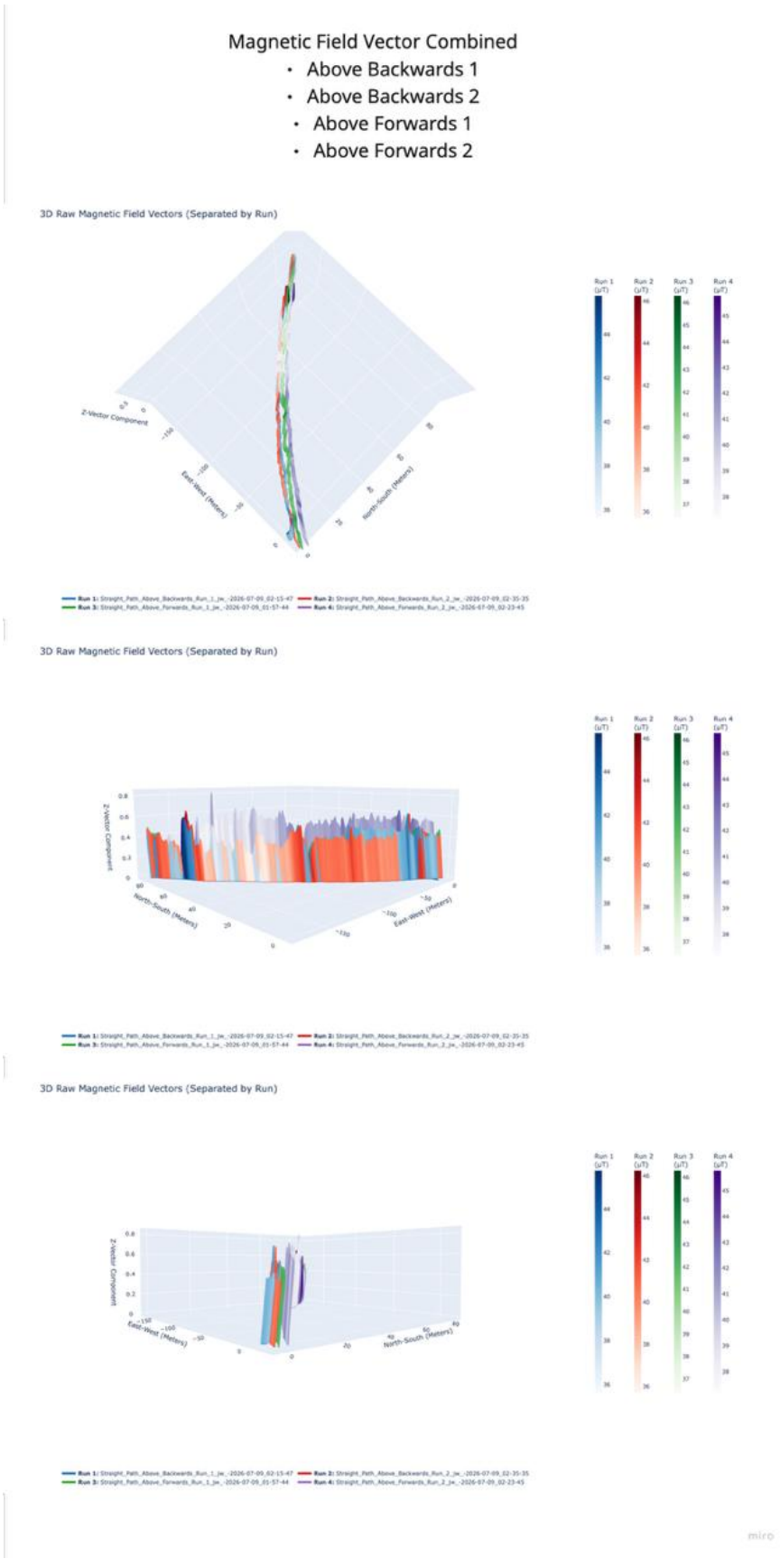


Figure 7: 3D spatial vector overlay combining all four trial runs along the "above" corridor.

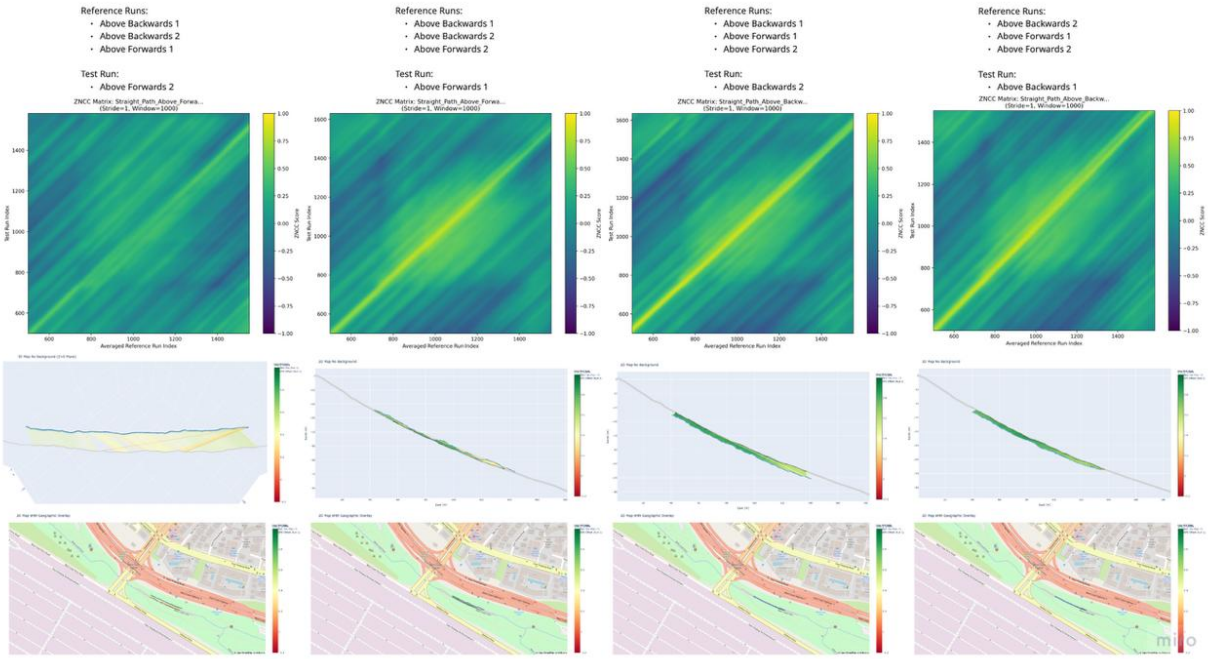


Figure 8: Pairwise ZNCC matrix plots and spatial location mappings across trial runs on the "above" corridor.

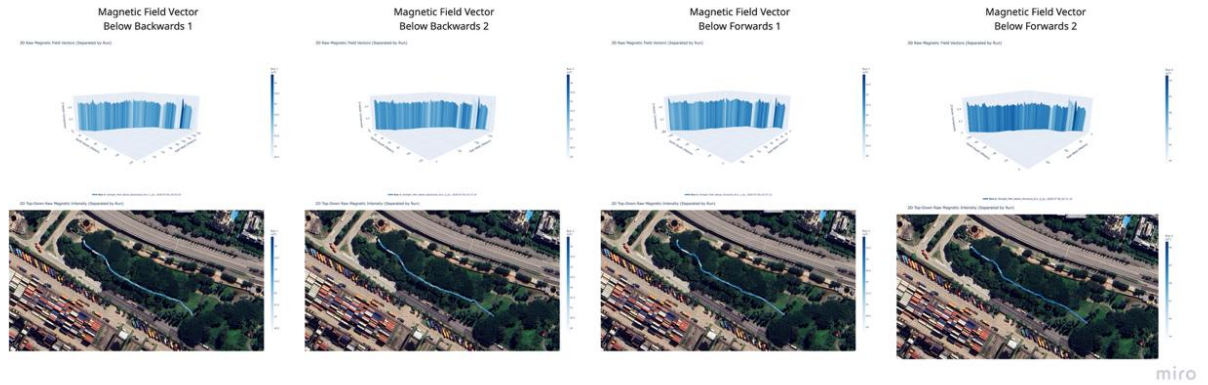


Figure 9: Individual 2D magnetic field vectors across the four passes on the "below" path.

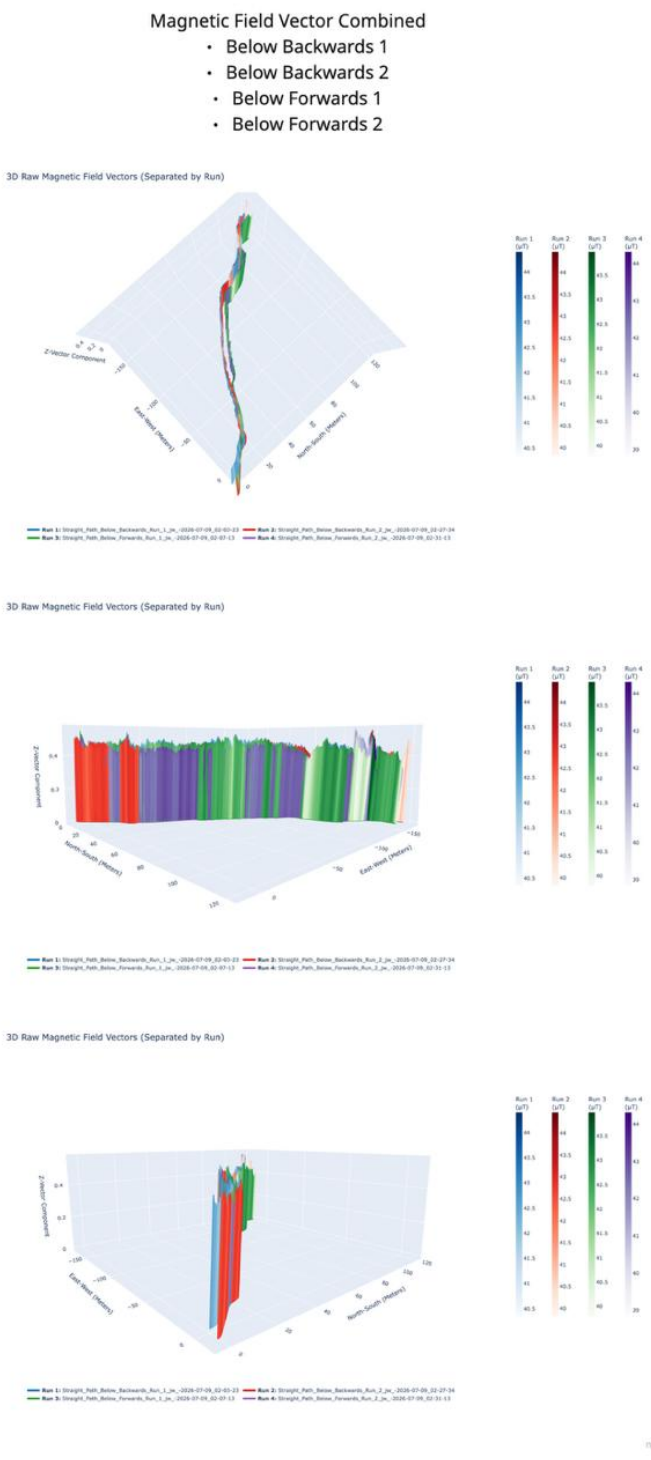


Figure 10: 3D spatial vector overlay combining all four trial runs on the "below" corridor.

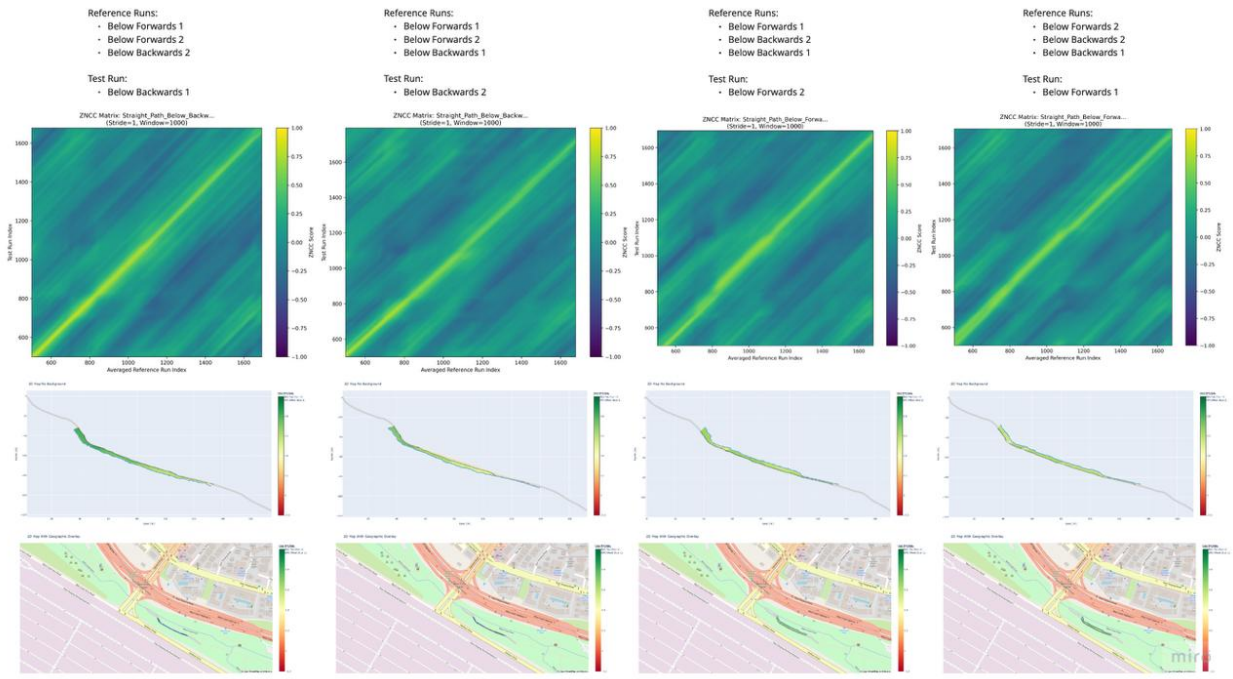


Figure 11: Pairwise ZNCC matrix plots and spatial location mappings for trial runs along the "below" path.

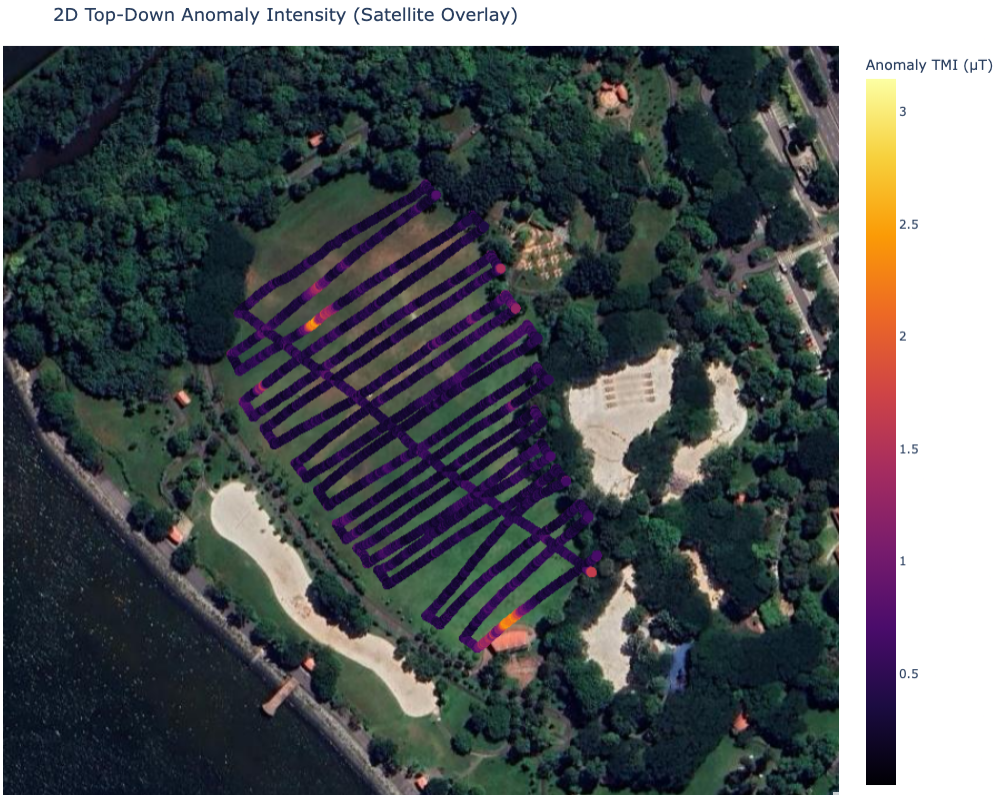


Figure 12: Top-down 2D map overlay displaying the spatial trajectory and Total Magnetic Intensity (TMI) recorded during the 40-minute lawn sweeping survey in front of the West Coast Park grandstand.

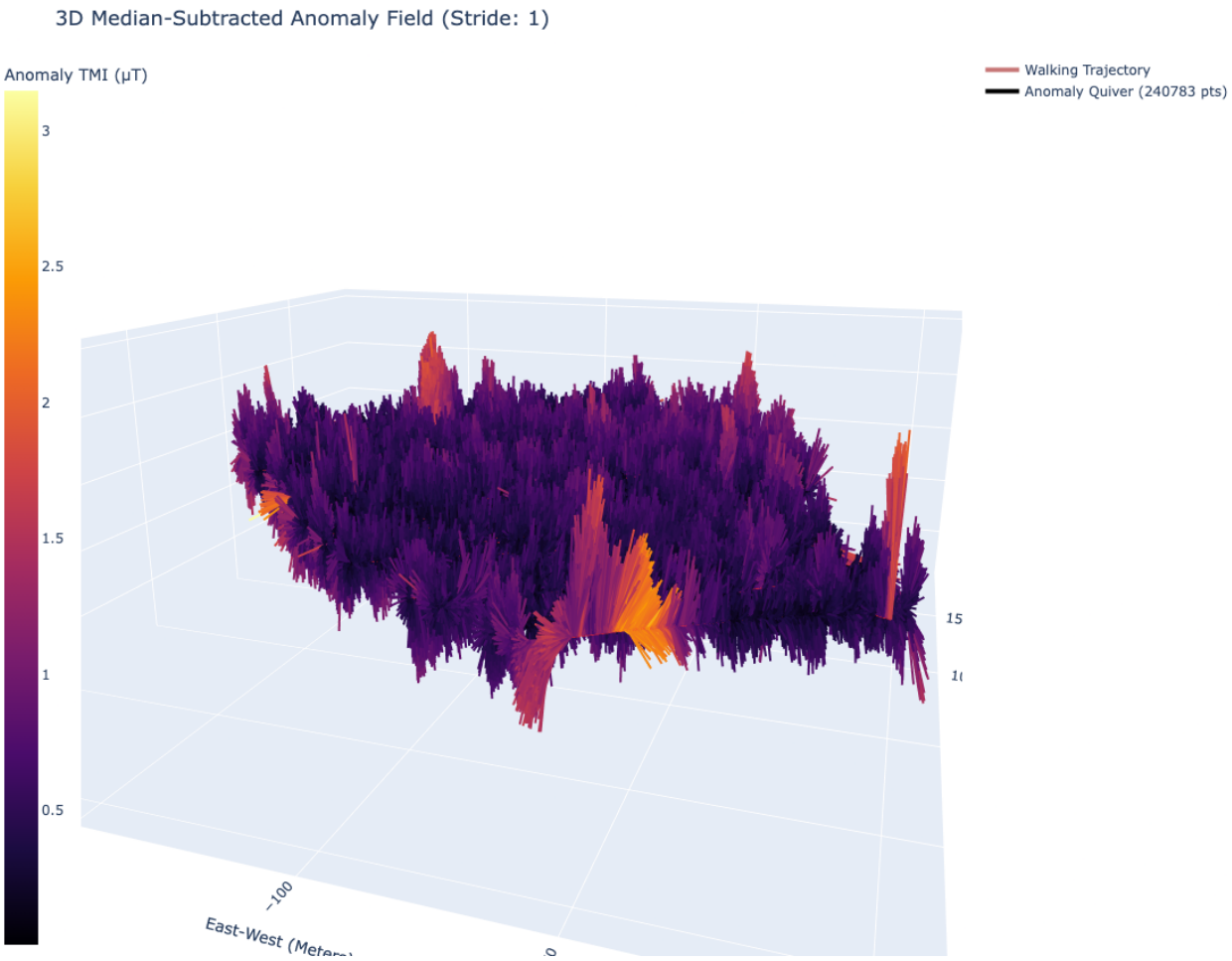


Figure 13: 3D spatial visualization of the median-subtracted magnetic anomaly vectors ($\mathbf{B}_n$) and interpolated scalar Total Magnetic Intensity (F) mapped across the survey grid.

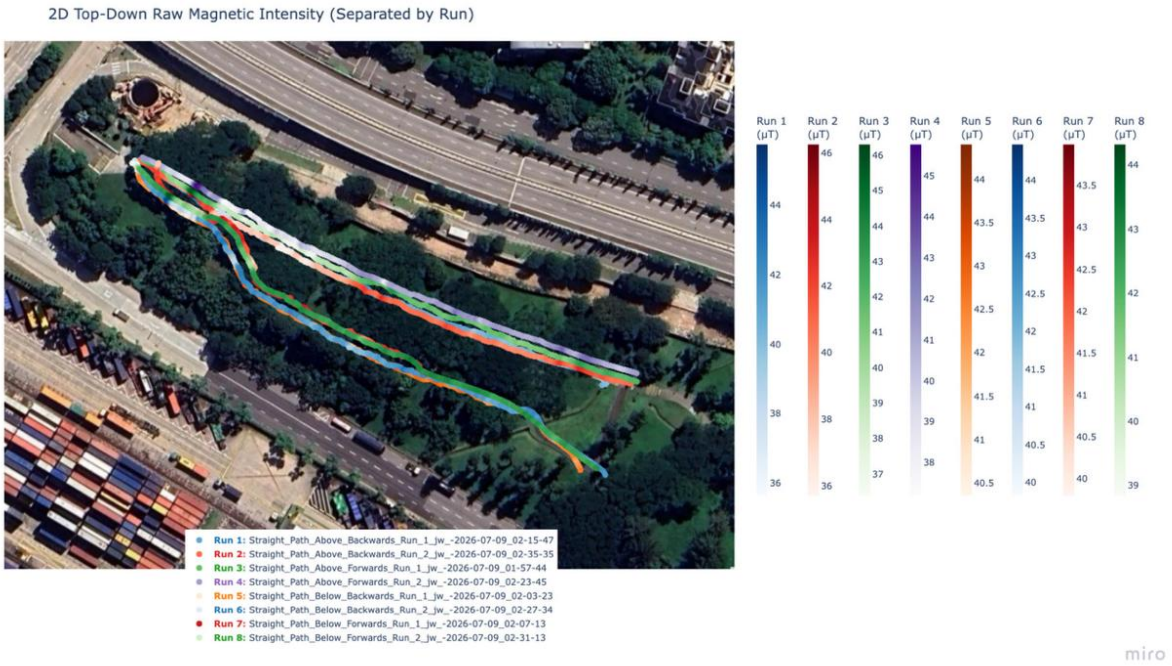


Figure 14: Combined scalar Total Magnetic Intensity (TMI) maps for both linear corridors overlaid on Google Maps coordinates.

9 Discussion

Overall, the magnetic field signatures exhibit strong spatial reproducibility across the experimental trials. This consistency is particularly evident in the 3D overlay of the raw magnetometer readings (Figure 15), where the spatial "fingerprint" or structural outline of the signal remains highly stable across multiple runs. Key features, such as sharp localised spikes and dips, recur consistently at identical physical locations along the straight paths.

However, a noticeable challenge is the presence of a drifting DC offset between consecutive runs. Even though all trial runs were completed within a narrow 10 to 15-minute timeframe, the absolute intensity (magnitude) of the magnetic field shifts noticeably between runs. Consequently, while the *shape* and relative feature locations of the magnetic field are highly repeatable, its absolute baseline value is not constant.

9.1 Above Path Performance and Anomaly Analysis

For the majority of the runs along the "above" path, the Zero-Normalised Cross-Correlation (ZNCC) yields strong, clear alignment, as indicated by a prominent yellow diagonal line across the correlation matrices. In these well-correlated scenarios, the normalised cross-correlation values consistently reach between 0.85 and 0.95.

The single notable exception occurs in one of the "above" backward runs (visualised in Figure 8), where the diagonal degraded significantly, yielding a peak cross-correlation score of only approximately 0.40. Tracing this degradation back to the raw signal plots in Figure 6, it becomes clear that the signal during this specific run suffered local distortion and failed to preserve the expected signature pattern. Crucially, the exact point along the spatial route where the location mapping begins to diverge corresponds precisely to the region of signal degradation, demonstrating a direct causal relationship between local signal fidelity and ZNCC tracking accuracy.

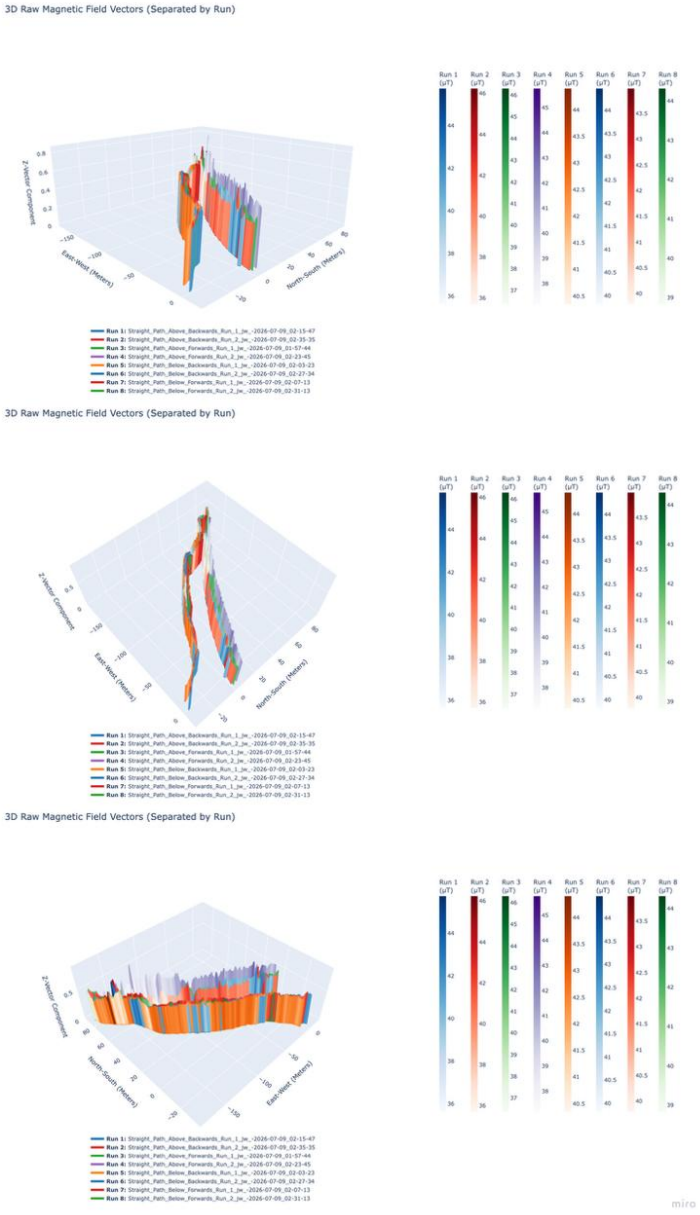


Figure 15: Combined 3D magnetometer vector overlay aggregating telemetry across all linear trial passes.

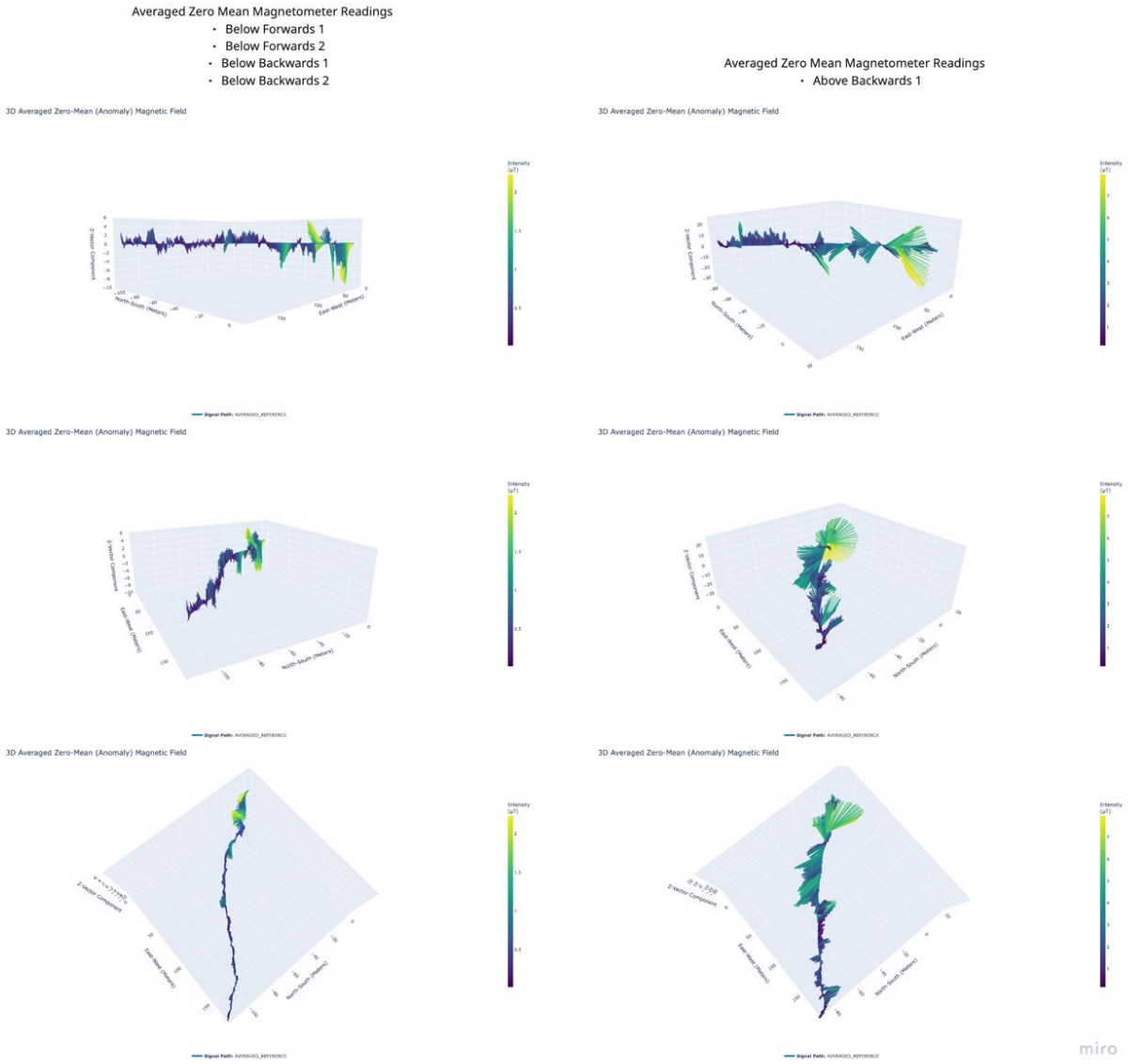


Figure 16: ZNCC signal matching for an "above" backward pass evaluated against a 3 : 1 half-reference pool.

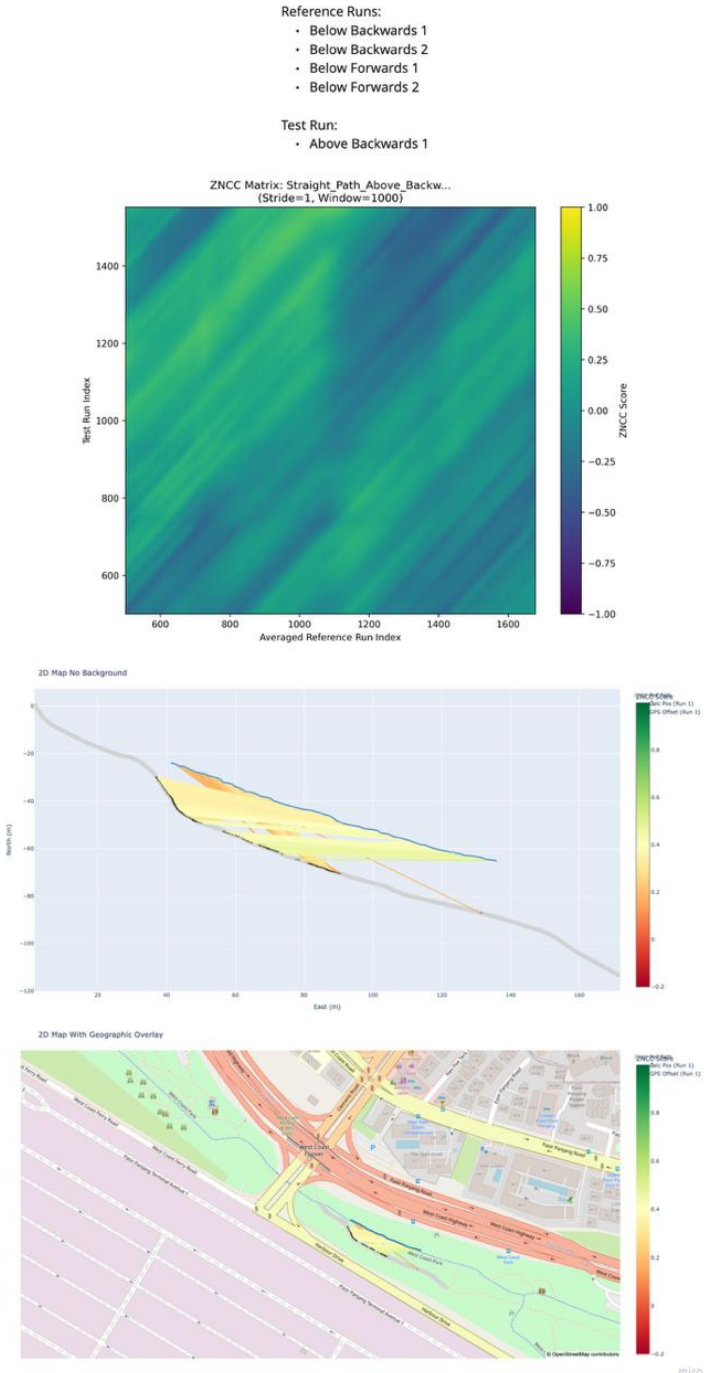


Figure 17: ZNCC correlation matrix for an "above" backward pass evaluated against a 3 : 1 half-reference pool.

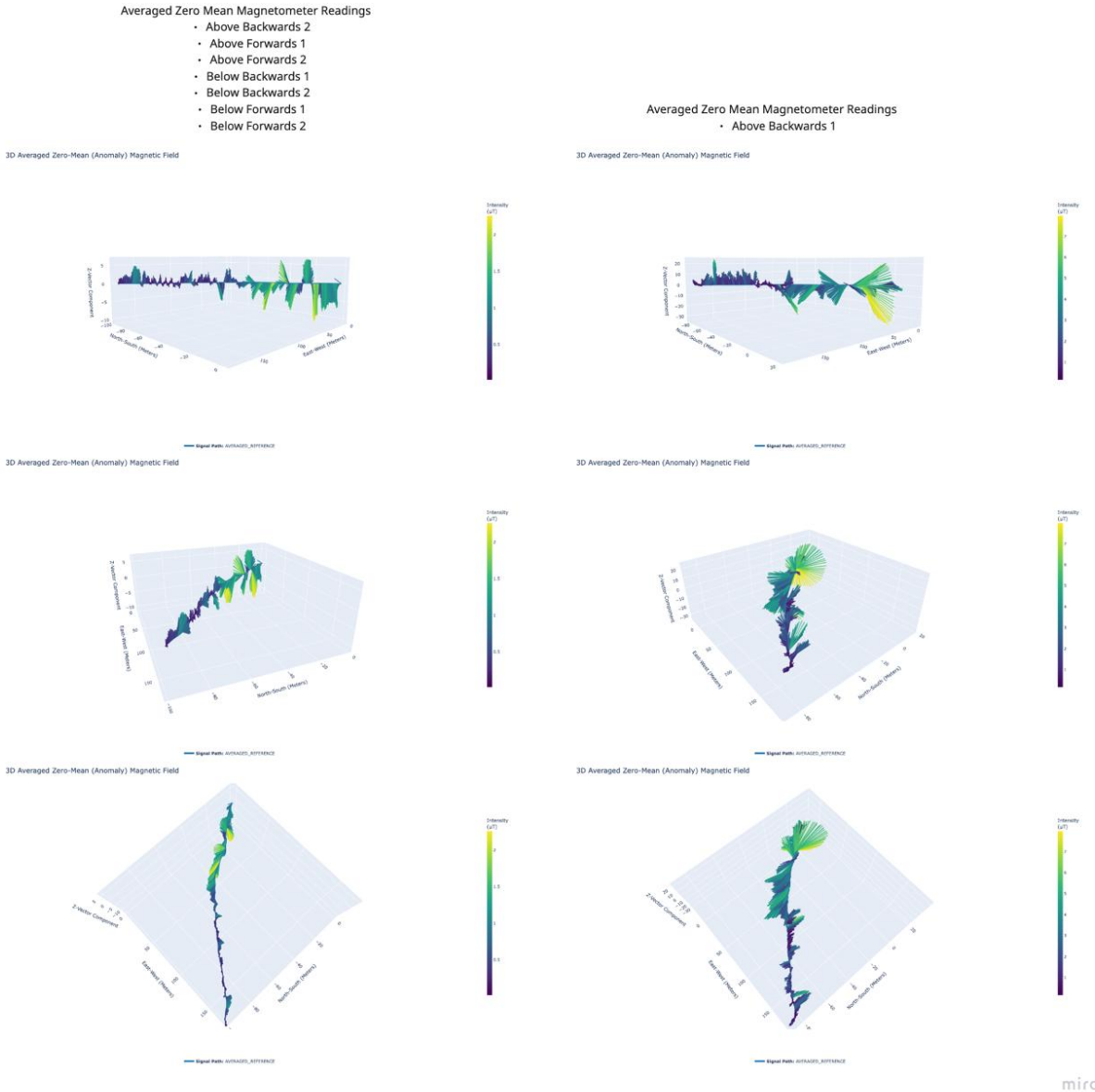


Figure 18: ZNCC signal matching for an "above" backward pass evaluated against a $7:1$ full-reference pool.

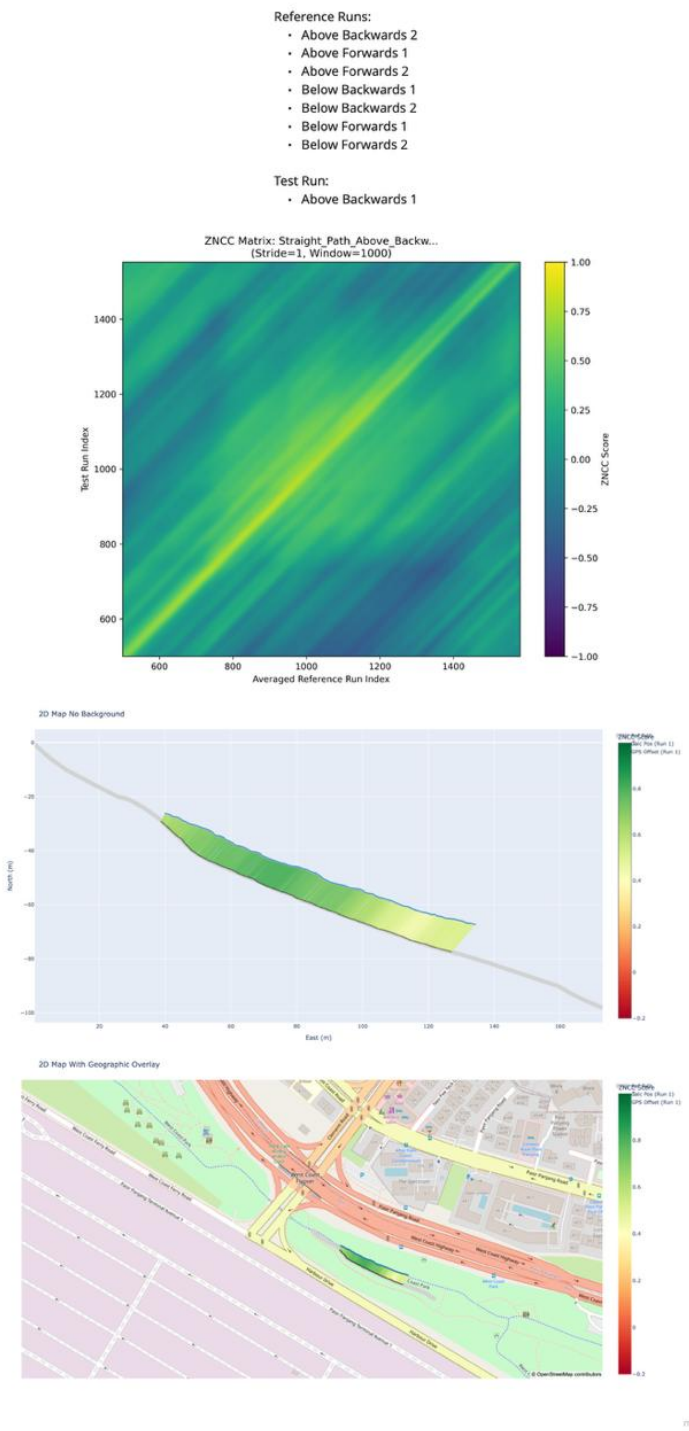


Figure 19: ZNCC correlation matrix for an "above" backward pass evaluated against a 7 : 1 full-reference pool.

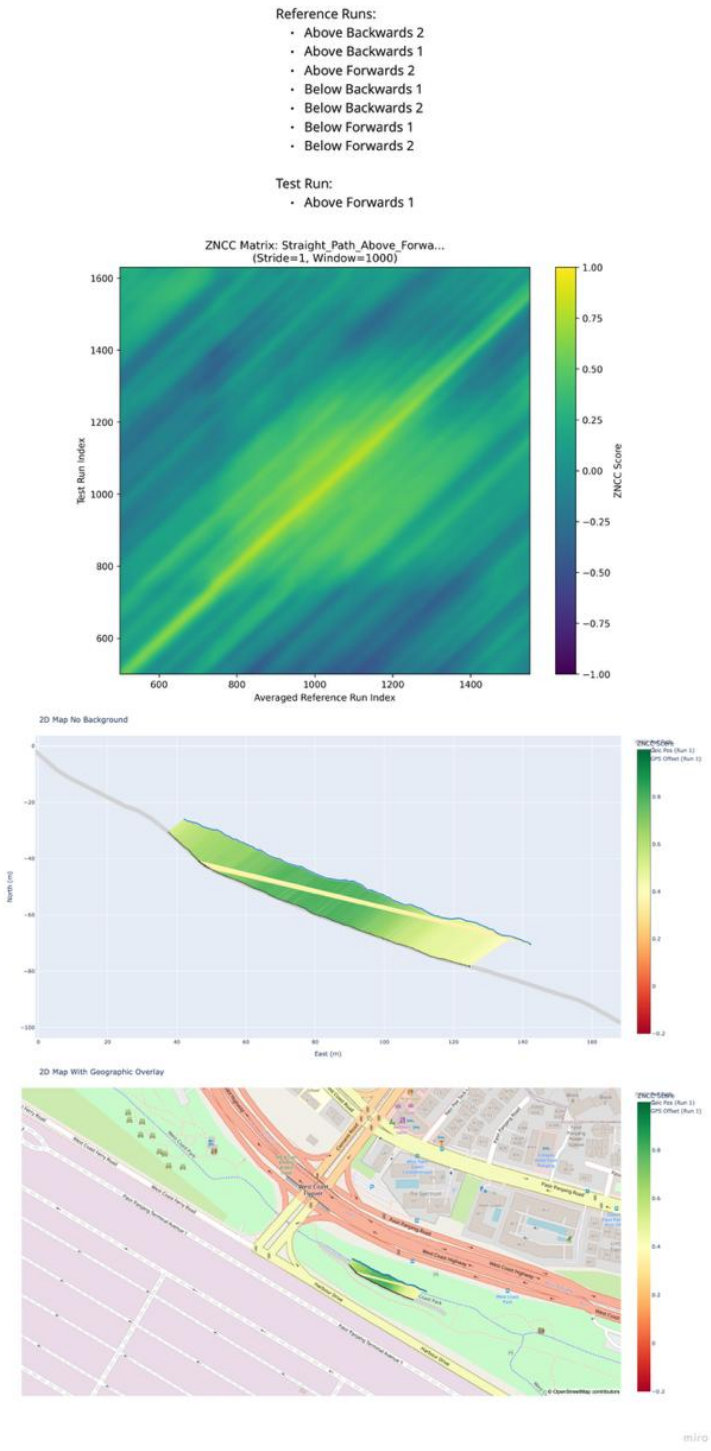


Figure 20: ZNCC test for "above" forward run 1 evaluated against the 7 : 1 full-reference map.

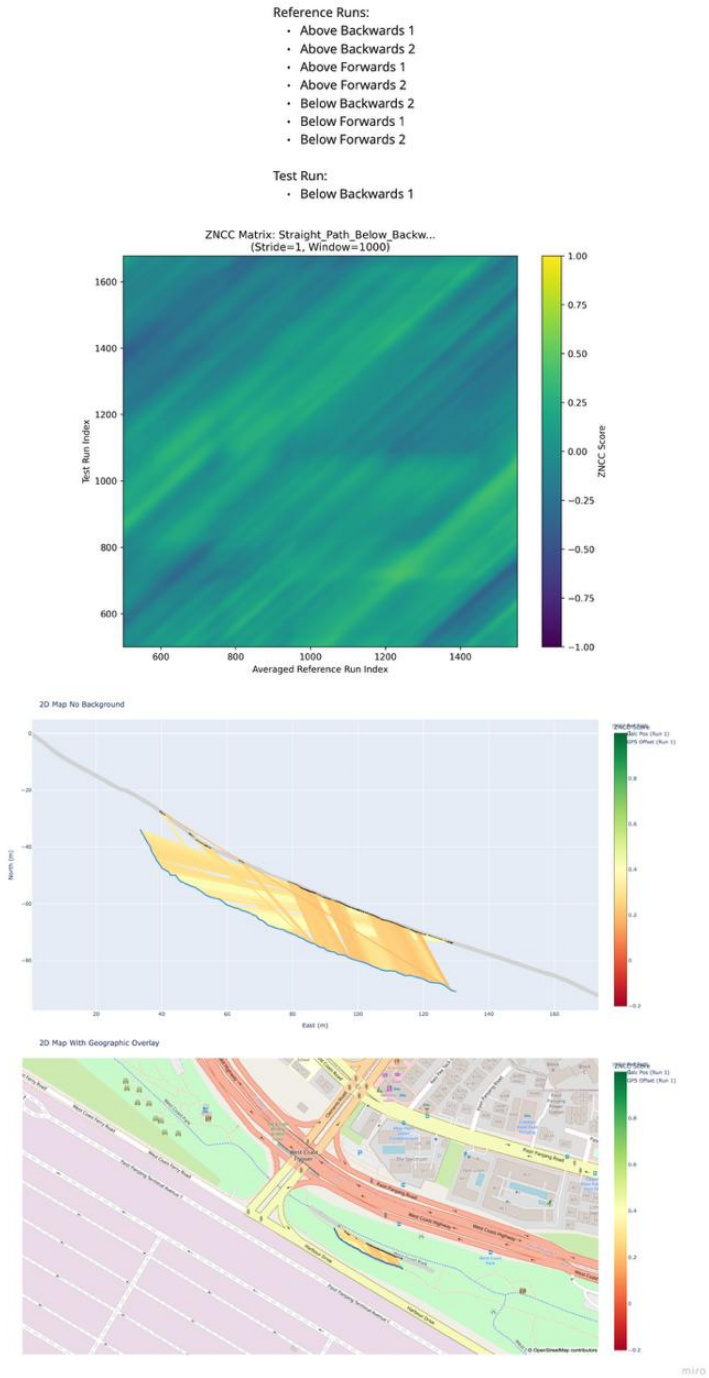


Figure 21: ZNCC test for "below" backward run 1 evaluated against the $7:1$ full-reference map.

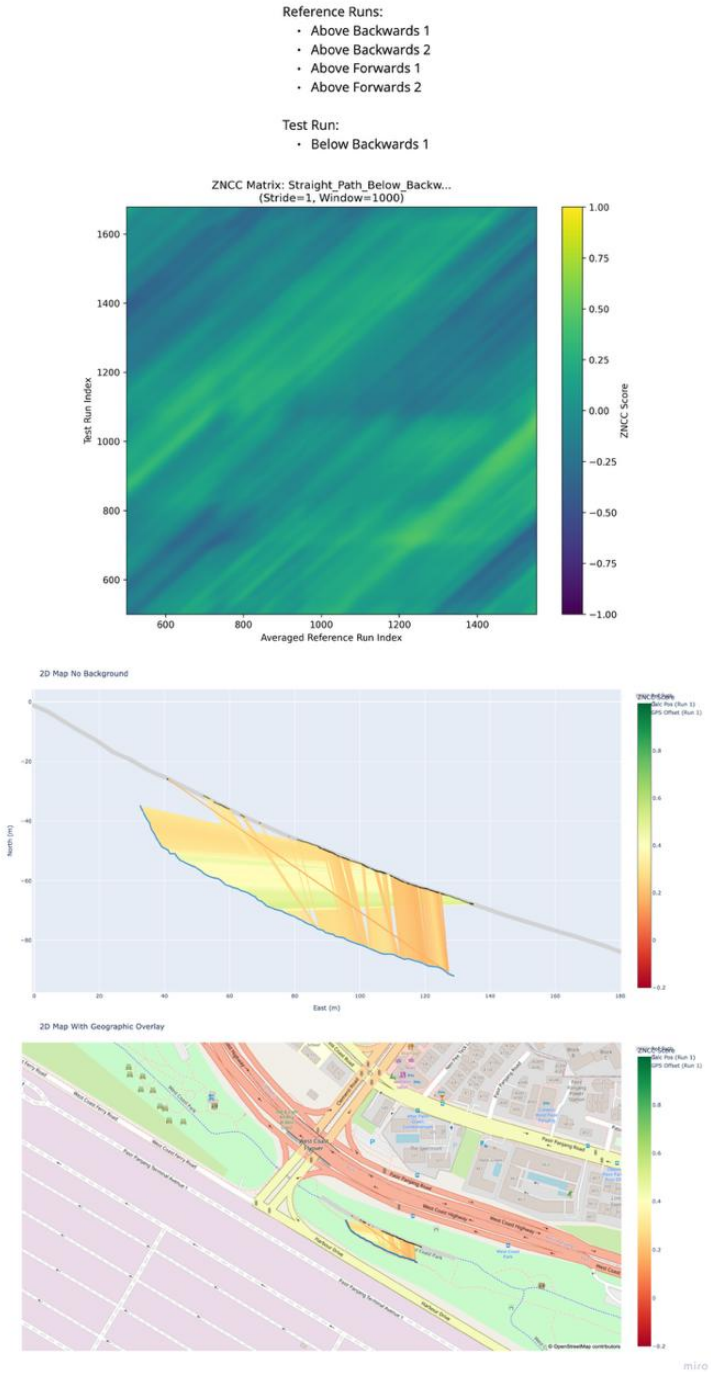


Figure 22: Cross-path ZNCC evaluation: a "below" backward pass tested against a 4 : 1 reference map composed purely of "above" path runs.

9.2 Below Path Performance

The findings from the "below" path closely mirror those of the "above" path. Signal fingerprinting remains sharp and distinct, while still exhibiting the same run-to-run DC offset drift in absolute field magnitude. Across all four trials on the "below" path, the ZNCC matrix maintains a strong yellow diagonal with correlation scores consistently hovering around 0.85 to 0.90, demonstrating robust signal repeatability along this route.

9.3 Implications for Kalman Filtering

These cross-correlation findings validate the viability of using magnetic anomalies for spatial matching, but they also highlight key considerations for future state estimation:

- **DC-Invariant Matching:** Because the absolute DC offset drifts over short timeframes, matching algorithms must rely on normalised or gradient-based features (such as ZNCC or spatial derivatives) rather than raw TMI intensity thresholds.
- **Dynamic Measurement Uncertainty:** In subsequent Kalman filter implementations, correlation quality (0.40 vs. 0.95) should dynamically scale the measurement covariance matrix R_k. When ZNCC correlation drops below a defined threshold, the filter can proportionally inflate the uncertainty assigned to magnetic position updates.
- **Adaptive Acceptance Thresholds:** Setting fixed or dynamically varying acceptance thresholds will allow the filter to reject corrupted signal segments (such as the degraded run observed on the "above" path) before they introduce trajectory drift into the estimation loop.

9.4 Spatial Uniqueness and Cross-Path Disambiguation

A primary objective of combining the dataset is evaluating whether paths in close spatial proximity (separated by less than 50 m) produce sufficiently distinct magnetic field signatures. As illustrated in the 3D vector overlay (Figure 15), the magnetic fingerprints of the two routes exhibit distinct spatial variations:

- **"Above" Path:** Demonstrates significantly higher local volatility with frequent, high-amplitude magnetic spikes along the trajectory.
- **"Below" Path:** Displays a smoother, more uniform baseline, though it retains subtle, highly repeatable local features sufficient for fingerprinting.

This distinct spatial variation confirms that localised man-made infrastructure and subtle environmental differences impart unique signatures to each path, preventing cross-route ambiguity.

9.5 Permutation Analysis: Reference Aggregation and Cross-Correlation Edge Cases

Across the various evaluation permutations (Figures 16 through 22), the cross-correlation results demonstrate both the power and limitations of reference averaging:

9.5.1 Positive Reference Behavior ($7:1$ Averaging and Expected Disambiguation)

- **Robustness of Averaged Maps:** Testing an individual run against a combined $7:1$ full-reference map (averaging both "above" and "below" paths) consistently yields strong correlation scores around 0.75 to 0.80, preserving a clear $1:1$ spatial mapping. This proves that merging signals across routes does not wash out key features into an unidentifiable signal; essential magnetic landmarks remain sharp and matching-capable.
- **Cross-Path Rejection:** As expected, attempting to match a test run from one path against a reference pool built exclusively from the opposing path (e.g., a "below" run matched against an "above" reference pool) results in poor correlation (≈ 0.40, seen in Figure 22). This confirms that the cross-correlation algorithm successfully discriminates between physically distinct routes.

9.5.2 Anomalies, Signal Quality Sensitivity, and Edge Failures

- **Local Spatial Divergence:** In certain 7 : 1 permutations (such as Figure 19), the majority of the trajectory maintains high correlation ($\approx$ 0.80–0.85), but localised signal degradation causes the tail end of the matching sequence to misalign by nearly 100 m, dropping the local ZNCC score to ≈ 0.50.
- **Unexpected Degradation in 7 : 1 Combinations:** In isolated instances, aggregating a 7 : 1 reference map resulted in surprisingly low correlation scores (≈ 0.30) for runs where high fidelity was anticipated. This highlights that reference aggregation remains sensitive to individual low-quality or corrupted constituent runs within the pool.
- **Spurious Partial Matching:** Conversely, during cross-path tests where poor correlation ($\approx$ 0.30–0.40) was expected throughout, certain local sub-segments exhibited unexpected partial matches reaching up to 0.60. These false positives underscore the necessity of evaluating trajectory trends rather than relying solely on isolated point correlations.

9.6 Windowing Constraints and Data Requirements

A key technical factor in these evaluations is the sliding correlation window size (100 m total length). Because no zero-padding was applied at signal boundaries, valid correlation outputs require a margin of $\pm$50 m forward and backward. As a result, usable trajectory matching initiates approximately one-quarter of the way into the path rather than at the absolute origin.

Ultimately, these empirical permutations demonstrate that while magnetic cross-correlation is fundamentally effective, its reliability is tightly coupled to individual test-run fidelity and reference selection. While anomaly occurrences and edge cases fluctuate across specific dataset configurations, expanding the volume of collected trial runs will progressively smooth out individual sensor anomalies and stabilize map construction.

9.7 2D Spatial Field Analysis and Discussion

The 2D spatial survey provides critical empirical insights into the urban magnetic environment, illustrating both the background field variations and the observational constraints of single-pass spatial mapping.

9.7.1 Environmental Noise vs. Sensor Sensitivity

A primary observation from the survey is that local spatial field variations occur on the order of several microteslas (μT). In contrast, modern quantum magnetometry platforms achieve sensitivity thresholds down to the nanotesla (nT) or sub-nanotesla regime. Consequently, in unshielded real-world environments, local spatial gradients and structural perturbations—rather than intrinsic sensor noise—serve as the dominant limiters of target detection and spatial field reconstruction.

9.7.2 Spatial Anomalies and Infrastructure Coupling

The total magnetic intensity (TMI) distribution across the survey grid exhibits distinct spatial features:

- **Grandstand Structural Anomalies:** Significant localized field distortion is observed along the perimeter adjacent to the West Coast Park grandstand. These spikes are attributed to structural soft-iron induction and hard-iron remanence from reinforced concrete footings, structural steel columns, and surface-level utility connections.
- **Interior Subsurface Gradients:** Isolated magnetic anomalies and local field spikes are also observed within the central lawn area. In the absence of accessible sub-surface engineering schematics, these spikes likely correspond to subterranean park infrastructure, such as drainage culverts, irrigation pipelines, or buried metallic debris.

9.7.3 Methodological Limitations and Dataset Utility

Because the survey was executed as a single 40-minute continuous logging run, temporal drift and exact spatial reproducibility cannot be independently isolated. However, the continuous, multi-directional coverage successfully captures representative two-dimensional spatial gradients across a continuous planar surface. These observed gradients validate the necessity of physics-informed spatial interpolation, providing the requisite spatial density to train and evaluate the Maxwell-conforming Gaussian Process Regression (GPR) kernel discussed in Section 5.

10 Future Work

While this work establishes a foundational baseline for consumer off-the-shelf (COTS) magnetic navigation, several practical limitations—including single-run 2D survey constraints, uncalibrated platform distortion, and coarse sensor sensitivity—highlight key avenues for future development. To transition from initial empirical mapping to a fully operational, real-time positioning system, future research will focus on the following roadmap.

10.1 Expanded 2D Field Surveys and Spatial Cross-Correlation

A primary priority is conducting extensive, multi-pass 2D spatial surveys across diverse geographic settings. While 1D corridors exhibited high Zero-Mean Normalised Cross-Correlation (ZNCC) alignment ($\gamma \approx 0.85$–0.95), verifying temporal stationarity across two dimensions requires repeated 2D sweeps over identical planar domains.

- **2D Correlation Formalism:** Developing robust 2D surface cross-correlation and image-registration techniques (e.g., 2D spatial cross-correlation or phase correlation) to quantify signal repeatability and map alignment across multi-pass planar sweeps.
- **Survey Domain Expansion:** Testing larger, unconstrained open areas beyond West Coast Park to determine the minimum spatial footprint and anomaly density required for unambiguous 2D position lock.

10.2 Gaussian Process Model Validation and Sensitivity Bounds

High-fidelity MagNav implementations in existing literature frequently rely on quantum magnetometers offering sub-nanotesla ($<$ nT) sensitivity. In contrast, smartphone magnetometers operate within the microtesla (μT) regime—a sensitivity gap of three orders of magnitude (10^3).

- **Signal vs. Noise Evaluation:** Further empirical testing is required to verify whether GPR interpolation on COTS datasets captures true spatial gradients ($\nabla \mathbf{B}$) or merely overfits high-frequency thermal noise floors.
- **GPR Cross-Validation:** Evaluating real-time cross-correlation directly against interpolated GPR lookup tensors ($\mathcal{T}_{\text{map}}$) to confirm that continuous spatial inference retains sufficient feature fidelity for arbitrary path matching.

10.3 Adaptive State Estimation and Filter Tuning

Deploying the Unscented Kalman Filter (UKF) in live scenarios requires dynamic adaptation to variable signal quality. As demonstrated in the 1D correlation analysis, local magnetic anomalies or transient interference can cause ZNCC scores to degrade significantly.

- **Dynamic Acceptance Thresholding:** Implementing adaptive gating mechanisms that raise or lower matching confidence thresholds based on real-time ZNCC peak quality.

- **Covariance Scaling:** Dynamically coupling the measurement covariance matrix (R_k) to live correlation scores, ensuring the filter automatically inflates measurement uncertainty during low-confidence or corrupted signal segments.
- **Parameter Refinement:** Conducting systemic parameter tuning to balance process noise (Q) against measurement update weighting, preventing filter divergence during extended GNSS blackout intervals.

10.4 Sensor Calibration and Advanced Preprocessing

Commercial smartphones are subject to onboard electromagnetic interference (EMI), temperature drift, and platform-induced distortions. Future iterations will incorporate rigorous sensor calibration protocols prior to map generation:

- **Hard/Soft Iron Calibration:** Executing standard three-axis figure-eight rotation maneuvers to model and remove permanent (hard-iron) and induced (soft-iron) magnetic distortions intrinsic to the device chassis.
- **Advanced Filtering and Literature Benchmarking:** Reviewing state-of-the-art preprocessing methodologies to better isolate environmental spatial gradients from sensor noise floors, establishing standard operational procedures for low-cost PNT platforms.

11 Conclusion

This report investigated the empirical feasibility of utilising commercial off-the-shelf (COTS) smartphone magnetometers for passive, unjammable magnetic anomaly navigation (MAGNAV) in Singapore's urban environment. Evaluated across 1D linear corridors and a 2D lawn sweeping grid at West Coast Park, the experimental findings yield three key conclusions:

1. **Spatial Repeatability and Uniqueness:** Despite microtesla-level (μT) urban background noise and short-term DC baseline drift across passes, rotated vector components ($\mathbf{B}_n$) and Total Magnetic Intensity (F) exhibit strong spatial repeatability along mapped paths, achieving Zero-Mean Normalised Cross-Correlation (ZNCC) scores between 0.85 and 0.95. Furthermore, parallel corridors separated by less than 20 m produced distinct spatial signatures, confirming that local infrastructure perturbations eliminate cross-route ambiguity.
2. **Physics-Informed Spatial Interpolation:** To bridge discrete survey logs and continuous navigation state estimation, we formulated a Maxwell-conforming Gaussian Process Regression (GPR) framework. By evaluating the double-del Hessian matrix operator over a latent scalar potential, the model enforces curl-free ($\nabla \times \mathbf{B} = \mathbf{0}$) and divergence-free ($\nabla \cdot \mathbf{B} = 0$) constraints, preventing unphysical field artifacts while generating dense, regularised lookup tensors ($\mathcal{T}_{\text{map}}$) with rigorous epistemic uncertainty bounds.
3. **Algorithmic Requirements for State Estimation:** The correlation analysis directly informs downstream Unscented Kalman Filter (UKF) design. Because raw magnetic baselines shift over time, estimation algorithms must rely on DC-invariant metrics (e.g., ZNCC or gradient observations) and incorporate dynamic covariance scaling ($R_k \propto 1/\gamma$) to downweight corrupted signal segments during simulated GNSS-denied intervals.

In summary, while consumer-grade mobile sensors lack the nanotesla sensitivity of quantum magnetometers and require explicit hard/soft-iron calibration, their ubiquity and ability to resolve spatially stable micro-anomalies demonstrate strong potential. Paired with physics-informed spatial regression and adaptive filtering, low-cost COTS magnetometry provides a viable foundation for resilient, ground-based positioning in GNSS-degraded urban environments.

12 Appendix A - Quaternions and DCM Derivation

12.1 Quaternions

Quaternions are an extension of how we used complex numbers to describe a 2D plane into 3D space. In 2D, we had the real part and the imaginary part, for which the combination of both of them allowed us to map any point in the 2D plane. We could describe rotations in 2D using the complex number i, where the angle of the complex number represented the angle of rotation. You might think that since we had 1 complex number and 1 real number to describe the 2D plane, we would need 2 complex numbers and 1 real number to describe the 3D plane. This however is false.

It is tempting to conclude that $a+ib$ looks very similar to (x, y) with the real part of the complex number being associated with the x axis and the imaginary part being associated with the y axis. Therefore for 3D, our incorrectly represented complex number might look something like $a+ib+jc$, each corresponding to the x, y, z axes respectively. However this is not the case. The confusion here stems from the fact that we are not using the number system to just represent coordinates, but we are using them to multiply and create rotations.

In 2D we only had 1 axis of rotation (the x-y plane), so we only needed 1 complex number. In 3D we have 3 axes of rotation (x-y, y-z, x-z planes), and hence we in fact require 3 complex numbers to represent the 3 axes of rotation. Since in 2D we had $e^{i\theta}$ where θ represents the angle of rotation about the x-y plane, it is easy to see that in 3D we would have $e^{i\theta_x}$, $e^{j\theta_y}$ and $e^{k\theta_z}$ where θ_x, θ_y and θ_z represent the angles of rotation about the x-y, y-z and x-z planes respectively. So there we have it, if we wanted to describe a rotation of any vector in the 3D plane, we need to know 6 things. How our axes are defined (x, y, z) and the angles (θ_x, θ_y, θ_z) to rotate the vector by, with respect to each axes.

Now, the genius of Leonhard Euler proved what is now referred to as the Euler's Rotation Theorem, that any arbitrary rotation in 3D space can be represented as a **singe** reflection about a single axis. I.e, if I wanted to rotate a vector from point A on a sphere to point B on the sphere, I only need to specify 2 things. The first being the plane of symmetry between the two points (from which we are able to describe a single vector that bisects the two points), and second (which can be derived from this single vector), the angle to which we want to rotate the first vector at point A by. Clearly this single axis of reflection may not be our unit axes, but in general will be a linear combination of them. I.e if we rotate about a diagonal axis that is equally weighted between the x-y and y-z planes, we can represent this as a single reflection about this diagonal axis. This is the key insight that allows us to represent any rotation in 3D space using just 1 angle, rather than more logically expressing 1 angle for each of the 3 axes of rotation. In other words, rather than thinking about rotating about x, then y, then z, we can think about looking at the final orientation and asking, what single custom diagonal line can I draw through space such that I only have to reflect my vector **once** by **one angle** to get it there?

This is the main conceptual insight and intuition behind quaternions. A very nice and concise way of describing an arbitrary rotation in 3D space with just 4 parameters, over the logical but cumbersome way of 6 parameters (3 angles and 3 axes). The full definition of a quaternion is as follows:

$$q = (q_w, \mathbf{v}) = q_w + q_x i + q_y j + q_z k = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ u_x \sin(\theta/2) \\ u_y \sin(\theta/2) \\ u_z \sin(\theta/2) \end{bmatrix} \tag{55}$$

notice that very nicely, we represent any arbitrary rotation axes (so that we can describe any arbitrary rotation with just one angle) using the u_x, u_y, u_z components, and we specify the specific angle of reflection using the θ component.

In so far as quaternions are concerned, we will be dealing with unit quaternions which satisfies the following:

$$\|q\| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} = 1 \tag{56}$$

The magnitude of a quaternion is akin to the determinant of a rotation matrix. We will be dealing with quaternions of unit 1, also called unit quaternions, which effectively preserve the magnitude of the vector that we are rotating.

12.2 Quaternion Operations

Now that we have our quaternion operator, we need to establish its natural operations. How do we add, conjugate and multiply the quaternion operator?

Addition is straightforward, we just add the components together:

$$q_1 + q_2 = (s_1 + s_2,\, \mathbf{v}_1 + \mathbf{v}_2) \tag{57}$$

Geometrically this is finding the average reflection of two quaternions, akin to resolving vectors via drawing a parallogram (hence the average rotation or "in between" of two quaternions) in hypecomplex space. After addition, the resultant quaternion would need to be normalised to result in a unit quaternion.

Conjugation is also straightforward, we just negate the vector part according to the usual complex conjugation rules:

$$q^* = (q_w,\, -\mathbf{v}) = q_w - q_x i - q_y j - q_z k \tag{58}$$

Geometrically this reverses the original reflection. If the original reflection was about some vector with some angle θ, then simply the conjugate of a quaternion is to reflect the vector about the same vector in question with the angle $-\theta$.

Multiplication is the operation which requires the most work but is geometrically intuitive to describe. It is simply the sequential application of quaternions. I.e $q_1 \otimes q_2$, where the $\otimes 0$ symbol is not the tensor product, but rather the standard notation for quaternion multiplication called the Hamilton product or grassmann product, is the sequential application of q_1 followed by q_2 to some vector. For this reason it should be immediately clear that Hamilton products are non-commutative.

To see this, we revisit how we define the multiplication rules for the imaginary units i, j and k. We know that $i^2 = j^2 = k^2 = -1$ from the analogy to complex numbers, but we also need to define how they interact with each other. Notice now that i, j and k represent axes in 3D space. If we take the cross product of two axes, we get a third axes that points in a specific direction. This should be nothing new from vectors, and recall that the sign of the cross product depends specifically on the order of the vectors. This is exactly what we see with the multiplication rules for i, j and k, clearly they must be non-commutative to capture this property of 3D space:

$$\begin{aligned} ij &= k, \quad ji = -k \\ jk &= i, \quad kj = -i \\ ki &= j, \quad ik = -j \end{aligned} \tag{59}$$

Now we have these non-commutative multiplication rules, we can investigate how to multiply two quaternion operators together. We can write the multiplication of two arbitrary quaternions q_1 and q_2 as follows:

$$q_1 = q_{1w} + q_{1x} i + q_{1y} j + q_{1z} k \tag{60}$$

$$q_2 = q_{2w} + q_{2x} i + q_{2y} j + q_{2z} k \tag{61}$$

$$q_1 \otimes q_2 = (q_{1w} + q_{1x} i + q_{1y} j + q_{1z} k)(q_{2w} + q_{2x} i + q_{2y} j + q_{2z} k) \tag{62}$$

Notice that taking the multiplication of the two quaternions requires distributing the terms, giving us in total 16 terms, and since the order of which the terms are multiplied matters, we cannot just group the i, j and k terms together. Expanding everything, grouping the terms and applying the multiplication rules for i, j and k gives us the following expansion, in matrix from for brevity:

$$q_1 \otimes q_2 = \begin{bmatrix} -q_{1x}q_{2x} - q_{1y}q_{2y} - q_{1z}q_{2z} \\ q_{2w}q_{1x} + q_{1y}q_{2z} - q_{1z}q_{2y} \\ q_{2w}q_{1y} - q_{1x}q_{2z} + q_{1z}q_{2x} \\ q_{2w}q_{1z} + q_{1x}q_{2y} - q_{1y}q_{2x} \end{bmatrix} \tag{63}$$

By inspection, we notice that the first row of Eq. 66 is nothing but the negative dot product of two vectors.

$$v_1 \cdot v_2 = -q_{1x}q_{2x} - q_{1y}q_{2y} - q_{1z}q_{2z} \tag{64}$$

Furthermore, using zero indexing, rows 1 to 3 and columns 1 to 3 of Eq. 66, by recognition, is nothing but the cross product of two vectors.

$$v_1 \times v_2 = \begin{bmatrix} q_{1y}q_{2z} - q_{1z}q_{2y} \\ q_{1z}q_{2x} - q_{1x}q_{2z} \\ q_{1x}q_{2y} - q_{1y}q_{2x} \end{bmatrix} \tag{65}$$

Therefore, we can summarise the effect of the grassmann product nicely into the following formula. You should check that the following formula matches all of the entries in the matrix given in Eq. (63).

$$q_1 \otimes q_2 = \big(q_{1w}q_{2w} - \mathbf{v}_1 \cdot \mathbf{v}_2, \quad q_{1w}\mathbf{v}_2 + q_{2w}\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2\big) \tag{66}$$

12.3 Direction Cosine Matrix (Direction Cosine Matrix) Derivation

Now that we have all the basic operations involving quaternions, we are ready to begin deriving the DCM. First recall that we are trying to describe the rotation of our iPhone's body frame (b-frame) against the global earth navigation frame (n-frame). Recall that our input control vector (which gives us acceleration in our b-frame) that we calculate in Eq. (21) is a 3D spatial vector. We want to calculate how much this 3D vector rotates overtime with respect to the n-frame, which we know since we are provided with the quaternion data by the Sensor Logger application. We also know how to rotate vectors using quaternions simply by applying the grassmann product! The issue we run into here is that our 3D vector is not of the same dimension as the quaternion. The trick here is to embed our 3D vector into 4D by assigning it a zero scalar component.

Say that our 3D spatial acceleration vector in the b-frame is:

$$\mathbf{a}_b = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \tag{67}$$

We embed $\mathbf{a}_b$ into hypercomplex space simply by assigning it a zero scalar component, which creates a pure spatial quaternion. Recall that the scalar value of a quaternion describes the angle of which the reflection occurs, while the remaining vector quantity describes the arbitrary vector to be reflected about. Hence a quaternion with zero angle is simply the axis vector to be rotated about, which in this case is simply our 3D acceleration vector, now represented in 4D space!

$$a_b = (0,\, \mathbf{a}_b) \tag{68}$$

You might think that okay, we are basically done right? If our quaternion data at time step k is q_k, simply apply the grassmann product, $a_b \otimes q_k$, to find how a_b has rotated right? Well unfortunately it is not that simple. Let's see what happens if evaluate the grassmann product according to Eq. 66.

$$a_b \otimes q_k = \begin{bmatrix} -\mathbf{a}_b \cdot \mathbf{v}_k \\ q_{kw}\mathbf{a}_b + \mathbf{a}_b \times \mathbf{v}_k \end{bmatrix} \tag{69}$$

However, notice that our scalar value $-\mathbf{a}_b \cdot \mathbf{v}_k$ is generally not zero, which implies that we have accidentally scaled our acceleration vector! So while we have correctly rotated it according to how our b-frame is rotating, we have accidentally modified its values. This cannot do, as we need to have them accurately represented, if not our acceleration in the n-frame will not be accurate.

To fix this, we need to somehow force the scalar value to be equal to zero while still applying the quaternion reflection via the grassmann product. Careful observation reveals to us that multiplying our immediate result with the conjugate of q_k allows us to achieve this. Let's evaluate what we call the "sandwich product" and investigate the final result. Applying q_k^* with the grassmann product of $(a_b \otimes q_k)$ gives us the following expansion:

$$a_n = q_k^* \otimes (a_b \otimes q_k) = \begin{bmatrix} q_{kw}(-\mathbf{a}_b \cdot \mathbf{v}_k) + \mathbf{v}_k \cdot (q_{kw}\mathbf{a}_b + \mathbf{a}_b \times \mathbf{v}_k) \\ q_{kw}(q_{kw}\mathbf{a}_b + \mathbf{a}_b \times \mathbf{v}_k) - (-\mathbf{a}_b \cdot \mathbf{v}_k)\mathbf{v}_k - \mathbf{v}_k \times (q_{kw}\mathbf{a}_b + \mathbf{a}_b \times \mathbf{v}_k) \end{bmatrix} \tag{70}$$

Distributing the operators across the internal brackets for each row yields:

$$a_n = \begin{bmatrix} -q_{kw}(\mathbf{a}_b \cdot \mathbf{v}_k) + q_{kw}(\mathbf{v}_k \cdot \mathbf{a}_b) + \mathbf{v}_k \cdot (\mathbf{a}_b \times \mathbf{v}_k) \\ q_{kw}^2\mathbf{a}_b + q_{kw}(\mathbf{a}_b \times \mathbf{v}_k) + (\mathbf{a}_b \cdot \mathbf{v}_k)\mathbf{v}_k - q_{kw}(\mathbf{v}_k \times \mathbf{a}_b) - \mathbf{v}_k \times (\mathbf{a}_b \times \mathbf{v}_k) \end{bmatrix} \tag{71}$$

Recall that dot products are commutative, cross products are anti-commutative and the result of the triple product, summarised below:

$$\mathbf{v}_k \cdot \mathbf{a}_b = \mathbf{a}_b \cdot \mathbf{v}_k \tag{72}$$

$$-\mathbf{v}_k \times \mathbf{a}_b = +\mathbf{a}_b \times \mathbf{v}_k \tag{73}$$

$$-\mathbf{v}_k \times (\mathbf{a}_b \times \mathbf{v}_k) = (\mathbf{v}_k \cdot \mathbf{v}_k)\mathbf{a}_b - (\mathbf{v}_k \cdot \mathbf{a}_b)\mathbf{v}_k = \|\mathbf{v}_k\|^2\mathbf{a}_b - (\mathbf{a}_b \cdot \mathbf{v}_k)\mathbf{v}_k \tag{74}$$

Applying these results we get the final simplification:

$$a_n = \begin{bmatrix} 0 \\ q_{kw}^2\mathbf{a}_b + 2q_{kw}(\mathbf{a}_b \times \mathbf{v}_k) + \|\mathbf{v}_k\|^2\mathbf{a}_b \end{bmatrix} = \begin{bmatrix} 0 \\ (q_{kw}^2 + |\mathbf{v}_k|^2)\mathbf{a}_b + 2q_{kw}(\mathbf{a}_b \times \mathbf{v}_k) \end{bmatrix} \tag{75}$$

Notice that we have achieved what we sought out to get. We wanted to describe how the $\mathbf{a}_n$ vector rotates according to the value of our quaternions. Very nicely we see that the scalar component of a_n is zero, leaving us with the raw result of our rotated acceleration vector $\mathbf{a}_n$. Now, we could stop here, however the final step would be to express this concise vector form into a matrix, which would ultimately yield our 3×3 Direction Cosine Matrix (DCM), denoted as R_b^n. I won't say much more, but let the derivation and mathematics speak for itself:

$$\mathbf{a}_n = (q_{kw}^2 + |\mathbf{v}_k|^2)\mathbf{a}_b + 2q_{kw}(\mathbf{a}_b \times \mathbf{v}_k) \tag{76}$$

$$\mathbf{a}_b = I_3\mathbf{a}_b = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \tag{77}$$

$$\mathbf{a}_b \times \mathbf{v}_k = -[\mathbf{v}_k] \times \mathbf{a}_b = \begin{bmatrix} 0 & q_{kz} & -q_{ky} \\ -q_{kz} & 0 & q_{kx} \\ q_{ky} & -q_{kx} & 0 \end{bmatrix} \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} \tag{78}$$

$$\mathbf{a}_n = \Big[(q_{kw}^2 + |\mathbf{v}_k|^2)I_3\Big]\mathbf{a}_b + \Big[-2q_{kw}[\mathbf{v}_k] \times \Big]\mathbf{a}_b \tag{79}$$

$$= \Big[(q_{kw}^2 + |\mathbf{v}_k|^2)I_3 - 2q_{kw}[\mathbf{v}_k] \times \Big]\mathbf{a}_b \tag{80}$$

$$(q_{kw}^2 + |\mathbf{v}_k|^2)I_3 = \begin{bmatrix} q_{kw}^2 + q_{kx}^2 + q_{ky}^2 + q_{kz}^2 & 0 & 0 \\ 0 & q_{kw}^2 + q_{kx}^2 + q_{ky}^2 + q_{kz}^2 & 0 \\ 0 & 0 & q_{kw}^2 + q_{kx}^2 + q_{ky}^2 + q_{kz}^2 \end{bmatrix} \tag{81}$$

$$-2q_{kw}[\mathbf{v}_k]\times = \begin{bmatrix} 0 & -2q_{kw}q_{kz} & 2q_{kw}q_{ky} \\ 2q_{kw}q_{kz} & 0 & -2q_{kw}q_{kx} \\ -2q_{kw}q_{ky} & 2q_{kw}q_{kx} & 0 \end{bmatrix} \tag{82}$$

$$R_b^n = \begin{bmatrix} q_{kw}^2 + q_{kx}^2 + q_{ky}^2 + q_{kz}^2 & -2q_{kw}q_{kz} & 2q_{kw}q_{ky} \\ 2q_{kw}q_{kz} & q_{kw}^2 + q_{kx}^2 + q_{ky}^2 + q_{kz}^2 & -2q_{kw}q_{kx} \\ -2q_{kw}q_{ky} & 2q_{kw}q_{kx} & q_{kw}^2 + q_{kx}^2 + q_{ky}^2 + q_{kz}^2 \end{bmatrix} \tag{83}$$

$$q_{kw}^2 + q_{kx}^2 + q_{ky}^2 + q_{kz}^2 = 1 \tag{84}$$

$$R_b^n = \begin{bmatrix} 1 - 2(q_{ky}^2 + q_{kz}^2) & 2(q_{kx}q_{ky} - q_{kw}q_{kz}) & 2(q_{kx}q_{kz} + q_{kw}q_{ky}) \\ 2(q_{kx}q_{ky} + q_{kw}q_{kz}) & 1 - 2(q_{kx}^2 + q_{kz}^2) & 2(q_{ky}q_{kz} - q_{kw}q_{kx}) \\ 2(q_{kx}q_{kz} - q_{kw}q_{ky}) & 2(q_{ky}q_{kz} + q_{kw}q_{kx}) & 1 - 2(q_{kx}^2 + q_{ky}^2) \end{bmatrix} \quad \text{Q.E.D.}$$

13 Appendix B - Gaussian Process Regression (GPR)

Refer to [12] for an intuitive understanding of GPR techniques.

13.1 Motivation

Our collected dataset $\mathcal{D}$ contains magnetic field values at specific positions in the n-frame that are entirely discrete. Utilising $\mathcal{D}$ wholesale to simulate GNSS-denied environments is not possible since a simulated trajectory may not physically intersect the exact data points in $\mathcal{D}$. How then do we determine the magnetic field values at unobserved locations?

For example, if we have a data point for the B-field at $(1, 1, 1)$ and a Kalman Filter on a subsequent run travels through $(1.2, 1.2, 1.2)$, should we assume the field at $(1.2, 1.2, 1.2)$ is identical to $(1, 1, 1)$? Alternatively, should we apply a decay value relative to our known B-field reading at $(1, 1, 1)$? The latter approach is inherently more realistic. Given that we possess numerous such B-field readings, the objective is to construct a smooth, continuous function $\mathbf{B}_{\text{map}}$ that can take any arbitrary 3D coordinate, $\mathbf{x} = [p_N, p_E, p_D]^T$, and return the corresponding B-field tensor (comprising the x, y, z B-field strengths in the n-frame, as well as the magnitude of the B-field vector) as described below:

$$\mathbf{B}_{\text{map}}(\mathbf{x}) = \begin{bmatrix} M_N(\mathbf{x}) \\ M_E(\mathbf{x}) \\ M_D(\mathbf{x}) \\ M_F(\mathbf{x}) \end{bmatrix} \tag{85}$$

But how do we do so in a manner that nicely accounts for the decay and change in the magnetic field? How do we reasonably extrapolate what an unknown B field reading would be when we have never mapped any data to that point?

13.1.1 Least Squares Regression

The simplest approach one might try would be to apply least squares regression to the dataset to generate a function of best fit as our continuous function. While this might seem like the simplest method to average out the values in $\mathcal{D}$, it introduces two key issues:

1. Model Assumptions: This method requires a pre-defined parametric model (e.g a polynomial of degree n), which introduces a high risk of overfitting or underfitting. Because we are modeling the magnetic fluctuations of a real-world environment, local disturbances such as vehicles, buildings, and structural steel, all contribute to the magnetic field readings. As such, the B-field readings are highly non-linear and can spike from region to region in a non obvious and unpredictable manner, so making the strong assumption at the beginning that the entire B field map can be determined with n degrees of freedom is bound to subject us to overfitting or underfitting, so this is not a good and robust strategy.

2. Lack of Spatial Uncertainty: For our Kalman Filter to function effectively, it requires a measure of uncertainty for each measurement. Applying a standard least squares regression on our dataset $\mathcal{D}$, while it yields a single deterministic value, that value does not come with any uncertainty. Think about how you might draw a line of best fit used to extrapolate data. You merely get a single point of what the value might be, but that value doesnt come with any inherent uncertainty. How might we get the uncertainty associated with any arbitrary point? You might suggest computing a global variance from the dataset, and using that single variance value to all points calculated along the least squares line. But clearly this is suboptimal, as uncertainty should vary depending on the Kalman Filter's current position relative to known data points. This makes sense right? If I happen to measure the B field at point $(1, 1, 1)$ in $\mathcal{D}$, and my Kalman Filter runs through the point $(1.01, 1.01, 1.01)$, I can conclude with a high certainty that the value should be very close to the value I got at point $(1, 1, 1)$. Conversely, if my Kalman Filter ran through point $(100, 100, 100)$, my uncertainty should be very high, since my only known point was incredibly far away, and hence I have no meaningful estimate of what my B field reading might be. So you see, my uncertainty has to vary depending on which arbitrary point I chose. Clearly doing a least squares regression alone will not give me the variance directly.

The solution that resolves both of these issues is Gaussian Process Regression (GPR).

13.2 Foundational Gaussian Process Theory

At its core, a Gaussian Process (GP) utilises the multivariate Gaussian Distribution. We have already seen multivariate Gaussian Distributions in the theory and implementation of continous Kalman Filtering. We will do a quick recap here. We are already familiar with a Gaussian Distribution, in one dimension, you only need 2 parameters to specify the entire shape of the distribution. The mean μ and the variance σ^2.

$$x \sim \mathcal{N}(\mu, \sigma^2) \tag{86}$$

$$\text{PDF}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{87}$$

Now we want to extend that principle up to three dimensions, since we have a position vector $\mathbf{x}_i = [x_i, y_i, z_i]^T \in \mathbb{R}^3$. So now rather than having a curve that varies in height along the 1D axis, we have a probability cloud floating in three dimensional space. But how do we describe variance σ^2 in three dimensions?

13.2.1 Variance and Covariance

Let's take a step back and understand what variance is really. Variance is the expected value we get when we take the expected difference of a variable about its own mean, squared.

$$\text{Var}(X_1) = \mathbb{E}\left[(X_1 - \mu_1)^2\right] \tag{88}$$

You might wonder why do we bother squaring it? If we want to know how much a variable varies about its own mean then simply taking

$$\text{Var}(X_1) = \mathbb{E}\left[(X_1 - \mu_1)\right] \tag{89}$$

should be enough no? Well, what happens if we get a symmetric difference above and below the mean? Say $X_1 = \pm 1$ and $\mu_1 = 0$, then we'll have a variance of 0, which means we conclude that well, our variable doesnt vary. So thats no good. We can try to do the modulus maybe?

$$\text{Var}(X_1) = \mathbb{E}\left[|(X_1 - \mu_1)|\right] \tag{90}$$

This seems to work, if I run my scenario again, $X_1 = \pm 1$ and $\mu_1 = 0$, I get a variance of 1, which is accurate to my original true variance! Now this measure of variance is referred to as **Mean Absolute Deviation (MAD)**, and is a perfectly good measure of variance. There are two reasons why we take the square over this method. Reason one, we want nice smooth differentiable functions for variance. Why do we want that? We'll see this later but take it on my word that we'll be differentiating our variance function (because we want to minimise variance to get the most accurate reading possible). **MAD** does not allow for smooth differentiating about 0, whereas the conventional definition of variance allows for it, so thats reason one. Reason two is more subtle. **MAD** is linear when it comes to describing variance, but squaring the difference makes it quadratic. In words, this punishes outliers quadratically more than **MAD** does. Having a point at 10 versus at 100, **MAD** says the variance is 10 and 100 respectively. Mean of squared deviations for the same 2 points says the variance is 100 and 10000, a factor of 10 for **MAD** and a factor of 100 for mean of squared deviations. So the latter is less trustful of data scatter. Why this is a better metric for a Kalman Filter and in statistics in general, I really don't know to be honest. But conventionally, mean of squared deviations is generally more widespreadly used.

Another way to see variance under the mean of squared deviations is to say that we are multiplying a single deviation by itself:

$$\text{Var}(X_1) = \mathbb{E}\left[(X_1 - \mu_1)(X_1 - \mu_1)\right] = \mathbb{E}\left[(X_1 - \mu_1)^2\right] \tag{91}$$

Now covariance asks a completely different question. Variance was asking how much does a variable fluctuate about its own mean. Covariance asks if I have two variables, when one fluctuates about its own center, how much and in what direction, does the other variable fluctuate about its own center?

$$\text{Cov}(X_1, X_2) = \mathbb{E}\left[(X_1 - \mu_1)(X_2 - \mu_2)\right] \tag{92}$$

The mention of direction indicates that covariance can either be negative, positive or 0. If $(X_1 - \mu_1)$ for example is positive, meaning we have gotten a value of X_1 that has fluctuated **above** its center, and say for example $(X_2 - \mu_2)$ is negative, meaning the reading of X_2 has fluctuated **below** its center, then overall, the covaraince will be negative. What does this imply? That both variables X_1 and X_2 fluctuate on average in opposite direction. It isn't too hard to see that if covariance is positive, then both X_1 and X_2 fluctuate in the same direction, either **above** or **below** on average. Now if covariance is 0, this means that on average, there is no clear direction in which X_1 and X_2 **correlate** in. There is no clear direction, could be **above** or **below** on average but those changes average out to 0. We say that a covariance of 0 implies that X_1 and X_2 are uncorrelated.

13.2.2 Multivariate Gaussian Distributions

Let's go back to answer our original goal, we want to extend the univariate Gaussian Distribution (what we typically just refer to the Gaussian Distribution involving only one variable) to three dimensions to match our position vector. The main issue was, how do we represent the variance σ^2 of this cloud in 3D space, and account for the possible correlations between variables?

We start by defining 3 independent, normally distributed variables z_1, z_2, z_3 which all have $\mu = 0, \sigma^2 = 1$. We will use these independent variables to define our correlated variables x, y, z. Think of z_1, z_2, z_3 as basis vectors; we cant define a vector without a basis vector, so any variable, correlated or not needs to begin with something, that something is a set of independently normally distributed variables. Now we describe x, y, z in terms of z_1, z_2, z_3, whose coefficients control the degree of correlation between x, y, z as a result.

$$\begin{aligned} x &= a_1 z_1 + a_2 z_2 + a_3 z_3 \\ y &= b_1 z_1 + b_2 z_2 + b_3 z_3 \\ z &= c_1 z_1 + c_2 z_2 + c_3 z_3 \end{aligned} \tag{93}$$

Recall the properties of distribution for mean, variance Eq. 88 and covariance Eq. 92, and linearity of the expectation $\mathbb{E}[\cdot]$ operator.

$$\begin{aligned} \mathbb{E}[a_1 z_1 + a_2 z_2 + a_3 z_3] &= a_1 \mathbb{E}[z_1] + a_2 \mathbb{E}[z_2] + a_3 \mathbb{E}[z_3] \\ \text{Var}(a_1 z_1 + a_2 z_2 + a_3 z_3) &= a_1^2 \text{Var}(z_1) + a_2^2 \text{Var}(z_2) + a_3^2 \text{Var}(z_3) \\ \text{Covar}(x, y) = \mathbb{E}[(x - \mu_x)(y - \mu_y)] &= \mathbb{E}[xy - x\mu_y - y\mu_x + \mu_x \mu_y] \\ = \mathbb{E}[xy] &= \mathbb{E}\big[(a_1 z_1 + a_2 z_2 + a_3 z_3)(b_1 z_1 + b_2 z_2 + b_3 z_3)\big] \end{aligned} \tag{94}$$

Substitute in the fact that $\text{Var}(z) = \sigma^2 = 1$, $\mu = 0$.

$$\begin{aligned} \text{Var}(x) &= a_1^2 + a_2^2 + a_3^2 \\ \text{Covar}(x, y) &= a_1 b_1 + a_2 b_2 + a_3 b_3 \end{aligned} \tag{95}$$

As such, generalising everything in matrix form:

$$\mathbf{X} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \mathbf{AZ} \tag{96}$$

By the definition of variance and complying with matrix multiplication rules:

$$\text{Var}(\mathbf{X}) = \mathbb{E}\left[(\mathbf{X} - \mu_{\mathbf{X}})^2\right] = \mathbb{E}\left[\mathbf{X}^2\right] = \mathbb{E}\left[\mathbf{X}\mathbf{X}^{\mathbf{T}}\right] \tag{97}$$

Substituting our transformation yields:

$$\begin{aligned} \text{Var}(\mathbf{X}) &= \mathbb{E}\big[(\mathbf{AZ})(\mathbf{AZ})^T\big] \\ &= \mathbb{E}\big[\mathbf{AZZ}^T\mathbf{A}^T\big] \end{aligned} \tag{98}$$

Since the coefficient matrices $\mathbf{A}$ and $\mathbf{A}^T$ are constants, we can pull them outside the expectation operator:

$$\begin{aligned}\text{Var}(\mathbf{X}) &= \mathbf{A}\,\mathbb{E}\big[\mathbf{Z}\mathbf{Z}^T\big]\,\mathbf{A}^T \\ &= \mathbf{A}\,\mathbb{E}\begin{bmatrix} z_1^2 & z_1z_2 & z_1z_3 \\ z_2z_1 & z_2^2 & z_2z_3 \\ z_3z_1 & z_3z_2 & z_3^2 \end{bmatrix}\mathbf{A}^T\end{aligned} \tag{99}$$

Because z_1, z_2, z_3 are independent, all their cross-terms are zero. Because they have a variance of 1, their squared terms are 1. Therefore, the variance matrix of our basis vector $\mathbb{E}[\mathbf{Z}\mathbf{Z}^T]$ is simply the identity matrix $\mathbf{I}$. This leaves us with our final matrix form:

$$\text{Var}(\mathbf{X}) = \mathbf{A}\mathbf{I}\mathbf{A}^T = \mathbf{A}\mathbf{A}^T \tag{100}$$

Finally, to prove this matches our algebraic expansion, we perform the explicit matrix multiplication of $\mathbf{A}$ and $\mathbf{A}^T$. Let $\mathbf{\Sigma}$ denote our final Variance-Covariance matrix:

$$\begin{aligned}\mathbf{\Sigma} = \text{Var}(\mathbf{X}) &= \mathbf{A}\mathbf{A}^T \\ &= \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}\begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \\ &= \begin{bmatrix} a_1^2+a_2^2+a_3^2 & a_1b_1+a_2b_2+a_3b_3 & a_1c_1+a_2c_2+a_3c_3 \\ a_1b_1+a_2b_2+a_3b_3 & b_1^2+b_2^2+b_3^2 & b_1c_1+b_2c_2+b_3c_3 \\ a_1c_1+a_2c_2+a_3c_3 & b_1c_1+b_2c_2+b_3c_3 & c_1^2+c_2^2+c_3^2 \end{bmatrix}\end{aligned} \tag{101}$$

By matching the elements of $\mathbf{\Sigma}$, we clearly see that the diagonal elements are precisely the variances while the off diagonal elements are the covariances between each pair of variables. Therefore:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} & \sigma_{xz} \\ \sigma_{yx} & \sigma_y^2 & \sigma_{yz} \\ \sigma_{zx} & \sigma_{zy} & \sigma_z^2 \end{bmatrix} \tag{102}$$

Now we can finally show our multivariate gaussian distribution, which takes the form of

$$\mathbf{r} \sim \mathcal{N}(\mu, \mathbf{\Sigma}) \tag{103}$$

which looks incredibly similar to our original form Eq. 86, except that our scalar numbers have been upgraded to vectors and matrices as shown by the boldface letters. We won't go through all of the linear algebra to show that the two are in fact functionally the same despite their difference in appearance, but we show you the similarities in structure for the univariate versus multivariate forms below. They don't look exactly the same but that is due to the difference in operations between scalars and matrices (or vectors). It isn't hard to see that because $(x - \mu_x)$ is a tall vector, squaring it requires us to take the transpose.

$$\text{PDF}(x) = \frac{1}{\sqrt{2\pi\sigma_x^2}}\exp\left(-\frac{1}{2}\frac{(x-\mu_x)^2}{\sigma_x^2}\right) \tag{104}$$

$$\text{PDF}(\mathbf{r}) = \frac{1}{\sqrt{(2\pi)^3|\mathbf{\Sigma}|}}\exp\left(-\frac{1}{2}(\mathbf{r}-\mu)^T\mathbf{\Sigma}^{-1}(\mathbf{r}-\mu)\right) \tag{105}$$

13.2.3 From Multivariate Gaussians to Gaussian Processes

While a multivariate Gaussian distribution defines the joint probability over a fixed, finite vector of random variables, a *Gaussian Process* (GP) extends this concept to an infinite collection of random variables indexed across a continuous space. To understand this monumental jump, we build up our framework systematically from our classic joint matrix convention in three steps.

Step 1: Scalar B Field Let us start with the simplest case. Suppose we temporarily ignore the directional vector components of the magnetic field and only track a single scalar property—its Total Magnetic Intensity (TMI), denoted as a scalar B.

If we want to track this scalar intensity at two distinct physical locations in our navigation environment simultaneously, say position $\mathbf{x} = [x_1, y_1, z_1]^T$ and position $\mathbf{x}' = [x_2, y_2, z_2]^T$,we have exactly two random variables: $B(\mathbf{x})$ and $B(\mathbf{x}')$. Using our established notation, $\text{Var}(\begin{bmatrix}\mathbf{x} & \mathbf{x}'\end{bmatrix}^T) = \mathbf{\Sigma}$ forms a simple 2×2 matrix composed of individual scalar entries:

$$\mathbf{\Sigma}_{\text{scalar-2pt}} = \begin{bmatrix} \sigma^2_{\mathbf{x}} & \sigma_{\mathbf{x},\mathbf{x}'} \\ \sigma_{\mathbf{x}',\mathbf{x}} & \sigma^2_{\mathbf{x}'} \end{bmatrix} \tag{106}$$

Notice that in the previous case we were talking about how each specific point was a variable, and hence had three components for the x, y and z axes, which meant that our $\mathbf{\Sigma}$ matrix had to be 3×3 in size. Here we only have 2×2 because we take the entire variable $\mathbf{x} \in \mathcal{R}^3$ as a random variable itself. Therefore, the diagonal terms in this new 2×2 $\mathbf{\Sigma}_{\text{scalar-2pt}}$ matrix represent the standalone variance at each isolated coordinate. The off-diagonal entry $\sigma_{\mathbf{x},\mathbf{x}'}$ describes the mutual spatial covariance between the two locations, i.e if $\mathbf{x}$ experiences a spike in intensity at its position, the covariance captures the expected correlated fluctuation at position $\mathbf{x}'$, consistent with the definition of covariance.

Step 2: Vector B Field Now, we bring back the full B field vector, which is a three dimensional vector, $\mathbf{B} = [B_N, B_E, B_D]^T$. Therefore, when we track the complete field at our two spatial positions $\mathbf{x}$ and $\mathbf{x}'$, we in essence need to track six random variables simultaneously, $\mathbf{B}_{\mathbf{x}} = [B_{\mathbf{x},N}, B_{\mathbf{x},E}, B_{\mathbf{x},D}]^T$ and $\mathbf{B}_{\mathbf{x}'} = [B_{\mathbf{x}',N}, B_{\mathbf{x}',E}, B_{\mathbf{x}',D}]^T$. Consequently, $\mathbf{\Sigma}$ matrix will also change to become a 6×6 sized matrix. We can see this change immediately by replacing the elements of the 2×2 scalar array with blocks of 3×3 submatrices:

$$\mathbf{\Sigma}_{\text{vector-2pt}} = \begin{bmatrix} \mathbf{\Sigma}_{\mathbf{x},\mathbf{x}} & \mathbf{\Sigma}_{\mathbf{x},\mathbf{x}'} \\ \mathbf{\Sigma}_{\mathbf{x}',\mathbf{x}} & \mathbf{\Sigma}_{\mathbf{x}',\mathbf{x}'} \end{bmatrix} \tag{107}$$

Where the entry blocks are parameterised explicitly using our directional sigma components:

$$\mathbf{\Sigma}_{\mathbf{x},\mathbf{x}} = \begin{bmatrix} \sigma^2_{B_N} & \sigma_{B_N,B_E} & \sigma_{B_N,B_D} \\ \sigma_{B_E,B_N} & \sigma^2_{B_E} & \sigma_{B_E,B_D} \\ \sigma_{B_D,B_N} & \sigma_{B_D,B_E} & \sigma^2_{B_D} \end{bmatrix}_{\text{at } \mathbf{x}}, \quad \mathbf{\Sigma}_{\mathbf{x},\mathbf{x}'} = \begin{bmatrix} \sigma_{B_N(\mathbf{x}),B_N(\mathbf{x}')} & \sigma_{B_N(\mathbf{x}),B_E(\mathbf{x}')} & \sigma_{B_N(\mathbf{x}),B_D(\mathbf{x}')} \\ \sigma_{B_E(\mathbf{x}),B_N(\mathbf{x}')} & \sigma_{B_E(\mathbf{x}),B_E(\mathbf{x}')} & \sigma_{B_E(\mathbf{x}),B_D(\mathbf{x}')} \\ \sigma_{B_D(\mathbf{x}),B_N(\mathbf{x}')} & \sigma_{B_D(\mathbf{x}),B_E(\mathbf{x}')} & \sigma_{B_D(\mathbf{x}),B_D(\mathbf{x}')} \end{bmatrix} \tag{108}$$

Take a minute to think about why this makes sense, specifically as to why it makes sense to replace the elements of the scalar block with 3×3 matrices. The variance of a scalar was simply computed by σ^2, but the variance of a vector $\begin{bmatrix}\mathbf{x} & \mathbf{x}'\end{bmatrix}^T$ would require a 2×2 $\mathbf{\Sigma}$ matrix, in order to capture how each variable correlated with each other. So having multiple variables, each being a vector, requires us to track how each variable varies with each other, in addition of how each component varies within itself in each variable! It isnt hard to conclude that the off-diagonal elements in $\mathbf{\Sigma}_{\text{vector-2pt}}$ accounts for the fluctuation along a certain axis say N at some location $\mathbf{x}$, with some other axis component say E at another completely different location $\mathbf{x}'$. So there we have it, we have variables varying with each other, and each component of each variable, also varying with every other component of each variable.

Step 3: Continuous Domains via Functional Kernels Here is the critical bottleneck for a real-world navigation system: as our vehicle moves continuously through space, it processes data along a time-varying trajectory containing an uncountably infinite number of potential coordinates. Under our old convention, if we wanted to build a joint matrix framework to map this space, we would be forced to manually define and tune unique correlation values between every single discrete index in our system. Attempting to assign static matrix indices to an infinite coordinate space is clearly impossible.

To resolve this limitation, we make a powerful simplifying assumption: we assume that the spatial covariance between the magnetic field at any two points in space is governed by a single equation. Instead of keeping a rigid, hardcoded lookup table of correlations, we introduce a continuous, coordinate-driven

function called a **kernel**, denoted as $k(\mathbf{x}), \mathbf{x}'$. This function acts as a dynamic matrix generator. It takes any two arbitrary spatial coordinates regardless of whether they were part of our original $\mathcal{D}$ or are completely unvisited points and outputs their exact covariance on the fly based purely on their geometric relationship. By shifting from static matrix indices to a universal functional equation, we can dynamically compile the exact joint covariance matrix needed for any arbitrary trajectory, whose final dimensions are bounded only by the number of active observations we choose to evaluate.

Squared Exponential Kernel To understand what this function actually looks like, we define its core engine using spatial proximity, also referred to as the **Squared Exponential (SE) Kernel**. In its baseline form, the cross-covariance between any two points decays exponentially as the geometric distance between them increases:

$$\mathbf{K}(\mathbf{x}, \mathbf{x}') = \mathbf{\Sigma}_{\mathbf{x},\mathbf{x}'} = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \cdot \mathbf{I}_3 \tag{109}$$

Where σ_f^2 represents the maximum signal variance, ℓ is a characteristic length-scale parameter dictating how quickly spatial correlations drop off, and $\mathbf{I}_3$ is a 3×3 identity matrix. Recall that $\mathbf{x}$ is a vector with 3 components, so we need to multiply this by a 3×3 identity so as to populate the diagonals, each one representing the variance of $B_N(\mathbf{x}, \mathbf{x}')$, $B_E(\mathbf{x}, \mathbf{x}')$ and $B_D(\mathbf{x}, \mathbf{x}')$ along the diagonals. If you are sharp you will call me out and say that the cross terms are empty, which means we are implying here that B_N, B_E, B_D all vary independently from each other. We will come back to fix this condition in the subsequent chapter, where we will also address what values ℓ and σ_f^2 should take on.

Okay, so now that we have given an actual function definition to our kernel, it is worth clarifying that while the input domain itself now has become continuous and infinite (we allow for any arbitrary point in space to be queried), the size of our $\mathbf{\Sigma}$ matrix is still limited by the dimension of $\mathcal{D}$. The kernel function $\mathbf{K}(\mathbf{x}, \mathbf{x}')$ is not a storage array for an infinite matrix, it is merely the generating function for all elements the $\mathbf{\Sigma}$ matrix. For example, suppose that in $\mathcal{D}$ we record $1,000$ discrete reference locations denoted as $\mathbf{X}_{\text{map}} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{1000}]^T$, then using our generative kernel function, we construct a massive, but strictly finite and trackable, 3000×3000 joint covariance matrix $\mathbf{\Sigma}_{\text{map}}$:

$$\mathbf{\Sigma}_{\text{map}} = \mathbf{K}(\mathbf{X}_{\text{map}}, \mathbf{X}_{\text{map}}) = \begin{bmatrix} \mathbf{K}(\mathbf{x}_1, \mathbf{x}_1) & \mathbf{K}(\mathbf{x}_1, \mathbf{x}_2) & \ldots & \mathbf{K}(\mathbf{x}_1, \mathbf{x}_{1000}) \\ \mathbf{K}(\mathbf{x}_2, \mathbf{x}_1) & \mathbf{K}(\mathbf{x}_2, \mathbf{x}_2) & \ldots & \mathbf{K}(\mathbf{x}_2, \mathbf{x}_{1000}) \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{K}(\mathbf{x}_{1000}, \mathbf{x}_1) & \mathbf{K}(\mathbf{x}_{1000}, \mathbf{x}_2) & \ldots & \mathbf{K}(\mathbf{x}_{1000}, \mathbf{x}_{1000}) \end{bmatrix} \tag{110}$$

We have come to the end of defining Gaussian Processes, and can summarise the GP relationship by stating that a vector valued Gaussian Process effectively defines a probability distribution over a continuous space of valid vector fields where $m(\mathbf{x}) : \mathbb{R}^3 \to \mathbb{R}^3$ represents the mean of the expected ambient B field:

$$\mathbf{B}(\mathbf{x}) \sim \mathcal{GP}\left(m(\mathbf{x}),\, \mathbf{K}(\mathbf{x}, \mathbf{x}')\right) \tag{111}$$

13.3 Real-Time Field Inference at an Arbitrary Point

Now that we have the formalism for the GPR, we can show what happens at run time. We will now address how do we make use of our surveyed points in $\mathcal{D}$ to make a reasonable inference for the value of $\mathbf{B}$ at some arbitrary point (that may not have been sampled in $\mathcal{D}$) using GPR. We do so via the principle of *marginal consistency*, which says that any finite snapshot we take of this continuous field will always behave as a predictable, standard Multivariate Gaussian distribution. In other words, we simply need to compute the giant $\mathbf{\Sigma}_{\text{grid}}$ matrix **once**, and it is valid to reuse that same block over and over again for any arbitrary point we want, as we don't expect the block to vary over time. This does not mean that it isnt physically possible for the B field to vary overtime (in reality the earth's B field is changing by the second), but here we make the assumption that the mapped B field is good enough for sequential runs and enough for us to repeatedly query it to get reliable estimates of the B field at new arbitrary points.

To compute the value of $\mathbf{B}(\mathbf{x}_*)$, we expand our Σ_{grid} structure into a joint matrix Σ_{joint} by evaluating the kernel's covariance values between the new query point and the existing survey network.

$$\boldsymbol{\Sigma}_{\text{joint}} = \begin{bmatrix} \mathbf{K}(\mathbf{X}_{\text{map}}, \mathbf{X}_{\text{map}}) & \mathbf{K}(\mathbf{X}_{\text{map}}, \mathbf{x}_*) \\ \mathbf{K}(\mathbf{x}_*, \mathbf{X}_{\text{map}}) & \mathbf{K}(\mathbf{x}_*, \mathbf{x}_*) \end{bmatrix} = \begin{bmatrix} \boldsymbol{\Sigma}_{\text{map}} & \boldsymbol{\Sigma}_{\text{map},\mathbf{x}_*} \\ \boldsymbol{\Sigma}_{\mathbf{x}_*,\text{map}} & \boldsymbol{\Sigma}_{\mathbf{x}_*,\mathbf{x}_*} \end{bmatrix} \tag{112}$$

Where:

- $\boldsymbol{\Sigma}_{\text{map},*} = \mathbf{K}(\mathbf{X}_{\text{map}}, \mathbf{x}_*) \in \mathbb{R}^{3000\times 3}$ is the cross-covariance block tracking the spatial correlation between the 1,000 baseline survey locations and the active query coordinate.
- $\boldsymbol{\Sigma}_{*,*} = \mathbf{K}(\mathbf{x}_*, \mathbf{x}_*) \in \mathbb{R}^{3\times 3}$ represents the intrinsic baseline variance block of the field evaluated at the isolated query coordinate.

To map these covariance structures to our recorded observations, we introduce our compiled empirical measurement vector, denoted as $\mathbf{Y}_{\text{map}} \in \mathbb{R}^{3000\times 1}$. This vector is a single column matrix constructed by vertically stacking our complete history of discrete 3D magnetic anomaly vectors captured across our 1,000 reference survey positions:

$$\mathbf{Y}_{\text{map}} = \begin{bmatrix} \mathbf{B}(\mathbf{x}_1) \\ \mathbf{B}(\mathbf{x}_2) \\ \vdots \\ \mathbf{B}(\mathbf{x}_{1000}) \end{bmatrix} = \begin{bmatrix} B_N(\mathbf{x}_1) \\ B_E(\mathbf{x}_1) \\ B_D(\mathbf{x}_1) \\ \vdots \\ B_D(\mathbf{x}_{1000}) \end{bmatrix} \tag{113}$$

With our joint system partitioned cleanly into our known historical data footprint $(\mathbf{X}_{\text{map}}, \mathbf{Y}_{\text{map}})$ and our unknown target coordinate $\mathbf{x}_*$, we apply standard Multivariate Gaussian linear conditioning identities to resolve our expected B field value.

To satisfy the assumption of a zero-mean Gaussian Process prior while maintaining physical realism, we explicitly compute an empirical mean $\mathbf{B}_0$ from the training data. Our prior mean function $m(\mathbf{x}_*)$ is set to this constant vector, and the GP operates on the mean-centered residuals, $\mathbf{Y}_{\text{residual}} = \mathbf{Y}_{\text{map}} - \mathbf{B}_0$.

$$\boldsymbol{\mu}_{\mathbf{B}}(\mathbf{x}_*) = \mathbf{B}_0 + \boldsymbol{\Sigma}_{\mathbf{x}_*,\mathbf{X}_{\text{map}}} \boldsymbol{\Sigma}_{\text{map}}^{-1} \mathbf{Y}_{\text{residual}} \tag{114}$$

$$\boldsymbol{\sigma}^2_{\mathbf{B}}(\mathbf{x}_*) = \boldsymbol{\Sigma}_{\mathbf{x}_*,\mathbf{x}_*} - \boldsymbol{\Sigma}_{\mathbf{x}_*,\mathbf{X}_{\text{map}}} \boldsymbol{\Sigma}_{\text{grid}}^{-1} \boldsymbol{\Sigma}_{\mathbf{X}_{\text{map}},\mathbf{x}_*} \tag{115}$$

Let's break down what these equations are doing. Eq. 114 takes in our mean-centered residuals $\mathbf{Y}_{\text{residual}}$, and $\boldsymbol{\Sigma}_{\text{grid}}$ contains all of the correlations between all points in the dataset. Inverting $\boldsymbol{\Sigma}_{\text{grid}}$ and applying that to $\mathbf{Y}_{\text{residual}}$ gives us a 3000×1 column of structural weights which we call $\boldsymbol{\alpha}$. Weights that when individually applied to $\mathbf{K}(\mathbf{x}_*, \mathbf{X}_{\text{map}})\boldsymbol{\alpha}$, in other words performing $\sum_{i=1}^{1000} \mathbf{K}(\mathbf{x}_*, \mathbf{x}_i)\boldsymbol{\alpha}_i$ gives us the weighted contribution of the value of the SE kernel at each point $\mathbf{x}_i$ in our dataset $\mathcal{D}$ with our queried point in question $\mathbf{x}_*$. Finally, the constant empirical mean $\mathbf{B}_0$ is added back to shift the prediction into the true global coordinate space.

13.4 Kernel Selection

As previously introduced, the standard choice for GPR is the Squared Exponential (SE) kernel, or Radial Basis Function (RBF) kernel in Eq. 109. We assumed simply that if two spatial survey points are close together in the Euclidean norm, their cross-covariance approaches the maximum signal variance σ_f^2. A measured anomaly at one point strongly updates our belief about its immediate spatial neighborhood, whereas points separated by distances much greater than the characteristic length-scale ℓ decouple completely.

While this geometric distance relationship perfectly models our scalar Total Magnetic Intensity (TMI) envelope $M_F(\mathbf{x})$, it introduces critical physical contradictions if naively stacked to model the directional vector components $\mathbf{B}(\mathbf{x}) = [B_N(\mathbf{x}), B_E(\mathbf{x}), B_D(\mathbf{x})]^T$. Recall that we are modelling magnetic B fields, hence we cannot naively assume that the scalar fields of the North, East and Down components are entirely decoupled.

Enforcing Maxwell's Field Equations In a localised, static navigation environment devoid of free macroscopic currents, the ambient macroscopic magnetic B-field must obey Gauss's law for magnetism and Ampère's circuital law for magnetostatics. This forces the vector field to strictly adhere to specific differential equations:

$$\nabla \cdot \mathbf{B} = 0 \quad \text{(Divergence-free constraint)} \tag{116}$$

$$\nabla \times \mathbf{B} = \mathbf{0} \quad \text{(Curl-free constraint)} \tag{117}$$

While magnetic fields universally possess non-zero curl in regions containing active current densities $\mathbf{J}$ (per Ampère's Law, $\nabla \times \mathbf{B} = \mu_0 \mathbf{J}$), because our tracking environment is bounded within the atmosphere, which is an electrical insulator where the localised free current density is effectively zero ($\mathbf{J} \approx \mathbf{0}$), the primary sources generating the ambient geomagnetic anomaly field (such as deep core geodynamo currents or crustal magnetisation) reside entirely outside our survey volume. Consequently, the field lines traversing our localised path propagate through an irrotational space. Any apparent non-zero curl observed in our training data can be confidently attributed to high-frequency sensor noise rather than local electrodynamics. Therefore, enforcing an exact curl-free constraint remains physically rigorous within our localised mapping domain.

If independent scalar kernels are used, the Gaussian Process predictive mean will inevitably draw physically impossible vector fields that generate fictitious magnetic monopoles, ghost currents, and localised flux leakage. To ensure our spatial reconstruction matches real-world electrodynamics, we must construct a matrix-valued kernel $\mathbf{K}_{\text{field}}(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^{3\times 3}$ whose off-diagonal blocks explicitly dictate the deterministic cross-talk between orthogonal axes based on Maxwell's equations.

Hessian Matrix Because any curl-free vector field can be expressed exactly as the gradient of a latent scalar magnetic potential field ($\mathbf{B} = \nabla\Phi$), we can analytically bake these differential constraints directly into our covariance operator. By fundamental vector calculus, any curl-free vector can be expressed as the gradient of an underlying latent scalar magnetic potential field $\Phi(\mathbf{x})$:

$$\mathbf{B}(\mathbf{x}) = \nabla\Phi(\mathbf{x}) \tag{118}$$

Hence, to model the spatial variations of this landscape, we place a zero-mean GP prior on the *fluctuations* of the latent potential field $\Phi(\mathbf{x})$ around our empirically derived constant mean:

$$\Phi(\mathbf{x}) \sim \mathcal{GP}\left(0, \text{Cov}(\Phi(\mathbf{x}), \Phi(\mathbf{x}'))\right) \tag{119}$$

We define the covariance of this latent potential using a scaled Squared Exponential (RBF) kernel:

$$\text{Cov}(\Phi(\mathbf{x}), \Phi(\mathbf{x}')) = \sigma_f^2 K(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \tag{120}$$

where $K(\mathbf{x}, \mathbf{x}')$ is the dimensionless geometric core kernel. ℓ is the characteristic length-scale (with units of meters, m). σ_f^2 is the signal variance of the latent potential, carrying the physical dimensions of $\text{Tesla}^2 \cdot \text{m}^2$. Recall that $\mathbf{B}\,[T] = \nabla\left[\frac{1}{m}\right]\Phi\,[\text{units of } \Phi] \implies \Phi\,[T \cdot m]$, and so the units of $\text{Cov}(\Phi(\mathbf{x}), \Phi(\mathbf{x}'))$ is $T^2 \cdot m^2$ as required.

Now since we want the covariance of the magnetic field $\mathbf{B}$, where $\mathbf{B}(\mathbf{x}) = \nabla\Phi(\mathbf{x})$, substituting these in to Eq. 120 gives us:

$$\text{Cov}(\mathbf{B}(\mathbf{x}), \mathbf{B}(\mathbf{x}')) = \text{Cov}\left(\nabla\Phi(\mathbf{x}), \nabla'\Phi(\mathbf{x}')\right) \tag{121}$$

Recall that since differentiation is a linear operator, ∇ behaves as a scalar. Following $\text{Cov}(aX, bY) = a \cdot b \cdot \text{Cov}(X, Y)$:

$$\text{Cov}(\mathbf{B}(\mathbf{x}), \mathbf{B}(\mathbf{x}')) = \nabla\left[\text{Cov}(\Phi(\mathbf{x}), \Phi(\mathbf{x}'))\right]\nabla'^T \tag{122}$$

Collecting the ∇ terms and substituting in the definition of $K(\mathbf{x}, \mathbf{x}')$ in Eq .120 produces:

$$\mathbf{K}_{\text{field}}(\mathbf{x}, \mathbf{x}') = \nabla\nabla'^T K(\mathbf{x}, \mathbf{x}') \tag{123}$$

where the double del operator ($\nabla\nabla'^T$) is the matrix notation for the Hessian matrix. In full form expanding our the matrix into vector form:

$$\mathbf{K}_{\text{field}}(\mathbf{x}, \mathbf{x}') = \frac{\sigma_f^2}{\ell^2} \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \left[\mathbf{I}_3 - \frac{(\mathbf{x} - \mathbf{x}')(\mathbf{x} - \mathbf{x}')^T}{\ell^2}\right] \tag{124}$$

13.5 Incorporating Homoscedastic Measurement Noise

In real-world survey tracking, our recorded measurement dataset $\mathcal{D}$ is fundamentally corrupted by high frequency instrumentation thermal noise and localised high frequency grid anomalies. Therefore, we cannot treat our observed training targets $\mathbf{Y}_{\text{map}}$ as perfect realisations of the underlying physical vector field $\mathbf{B}(\mathbf{x})$. Instead, we model each discrete observation entry as:

$$\mathbf{y}_i = \mathbf{B}(\mathbf{x}_i) + \boldsymbol{\epsilon}_i \tag{125}$$

where $\boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \sigma_n^2 \mathbf{I}_3)$ represents an independent, identically distributed (i.i.d.) homoscedastic sensor measurement noise vector.

When stacking all N discrete training samples into our full multi-variable system, this noise variance only manifests when a survey point is correlated with itself (i.e., along the absolute diagonal entries of our global joint covariance matrix). For two distinct spatial points $\mathbf{x}_i \neq \mathbf{x}_j$, the measurement noise terms are completely uncorrelated ($\mathbb{E}[\boldsymbol{\epsilon}_i \boldsymbol{\epsilon}_j^T] = \mathbf{0}$). Consequently, the full multi-variable covariance operator $\mathbf{K}_y \in \mathbb{R}^{3N \times 3N}$ is constructed by adding a diagonal block noise matrix to our noise-free analytical kernel evaluations:

$$\mathbf{K}_y = \mathbf{K}(\mathbf{X}_{\text{map}}, \mathbf{X}_{\text{map}}) + \sigma_n^2 \mathbf{I}_{3N} \tag{126}$$

To fully comprehend the structural dimensions of this system, consider the partitioned structure of $\mathbf{K}_y$. Because each spatial coordinate interaction maps to a 3×3 cross-covariance submatrix generated by our Maxwell-conforming field Hessian, the comprehensive training matrix forms a block matrix of size $N \times N$ blocks:

$$\mathbf{K}_y = \begin{bmatrix} \mathbf{K}_{\text{field}}(\mathbf{x}_1, \mathbf{x}_1) + \sigma_n^2 \mathbf{I}_3 & \mathbf{K}_{\text{field}}(\mathbf{x}_1, \mathbf{x}_2) & \dots & \mathbf{K}_{\text{field}}(\mathbf{x}_1, \mathbf{x}_N) \\ \mathbf{K}_{\text{field}}(\mathbf{x}_2, \mathbf{x}_1) & \mathbf{K}_{\text{field}}(\mathbf{x}_2, \mathbf{x}_2) + \sigma_n^2 \mathbf{I}_3 & \dots & \mathbf{K}_{\text{field}}(\mathbf{x}_2, \mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{K}_{\text{field}}(\mathbf{x}_N, \mathbf{x}_1) & \mathbf{K}_{\text{field}}(\mathbf{x}_N, \mathbf{x}_2) & \dots & \mathbf{K}_{\text{field}}(\mathbf{x}_N, \mathbf{x}_N) + \sigma_n^2 \mathbf{I}_3 \end{bmatrix} \tag{127}$$

The identity matrix $\mathbf{I}_{3N}$ thus scales cleanly across the full $3N$ dimensions, ensuring that the inversion of $\mathbf{K}_y$ during real-time inference remains numerically stable, regularized, and protected against singular matrices resulting from identical co-located survey coordinates.

13.6 Mathematical Deconstruction of Offline Hyperparameter Optimisation

The optimal hyperparameter vector $\boldsymbol{\theta}^* = \{\ell, \sigma_f^2, \sigma_n^2\}$ defines the spatial behavior and regularisation constraints of our physical vector map. Because a Gaussian Process framework treats the training dataset targets $\mathbf{y}$ as a single joint draw from a high-dimensional Multivariate Gaussian distribution, the probability density of observing our specific survey data given a trial hyperparameter configuration $\boldsymbol{\theta}$ is defined exactly by the generative likelihood function:

$$p(\mathbf{y} \mid \mathbf{X}_{\text{map}}, \boldsymbol{\theta}) = \frac{1}{\sqrt{(2\pi)^{3N} |\mathbf{K}_y|}} \exp\left(-\frac{1}{2} \mathbf{y}^T \mathbf{K}_y^{-1} \mathbf{y}\right) \tag{128}$$

To convert this into a numerically stable optimisation objective, we take the natural logarithm of the likelihood function. Maximizing this log probability yields the objective outlined in Eq. 128, which expands into three distinct, competing mathematical operators:

$$\log p(\mathbf{y} \mid \mathbf{X}_{\text{map}}, \boldsymbol{\theta}) = \underbrace{-\frac{1}{2} \mathbf{y}^T \mathbf{K}_y^{-1} \mathbf{y}}_{\text{Data Fit Term}} - \underbrace{\frac{1}{2} \log \det(\mathbf{K}_y)}_{\text{Complexity Penalty}} - \underbrace{\frac{3N}{2} \log(2\pi)}_{\text{Normalization Constant}} \tag{129}$$

Analyzing the internal mechanics of these terms highlights how the optimization balances data fidelity against structural overfitting:

- **The Data Fit Term ($-\frac{1}{2}\mathbf{y}^T\mathbf{K}_y^{-1}\mathbf{y}$):** This quadratic form is the only term that actively looks at the empirical magnetic field readings contained in $\mathbf{y}$. It measures how well the chosen kernel structure explains the actual training data. If the length-scale ℓ is set too large, or if the signal variance σ_f^2 is vastly underestimated, $\mathbf{K}_y^{-1}$ penalises the model severely for failing to match the recorded spatial variations, causing this term to drop precipitously.
- **The Complexity Penalty ($\frac{1}{2}\log\det(\mathbf{K}_y)$):** This term acts as a rigid Occam's razor regularisation filter. It depends solely on the covariance matrix geometries, entirely independent of the measurement values in $\mathbf{y}$. If the optimisation attempts to overfit the local high-frequency sensor noise by shrinking the length-scale $\ell \to 0$, the correlation entries collapse, making $\mathbf{K}_y$ look like a highly flexible identity matrix. This drastic increase in model capacity causes the determinant $\det(\mathbf{K}_y)$ to explode, heavily punishing the objective function.
- **The Normalisation Constant ($\frac{3N}{2}\log(2\pi)$):** This term is a fixed geometric constant scaling with the total dimensionality ($3N$) of our stacked vector space, ensuring that the continuous underlying integral integrates perfectly to a valid total probability of 1.

By utilising a non-linear optimiser (such as the safe Quasi-Newton L-BFGS-B algorithm) to maximize this joint objective, the framework naturally balances these competing forces to extract the exact physical correlation parameters that best explain the localised magnetic field landscape without overfitting sensor imperfections.

13.7 Numerical Block Interleaving and Vectorised Matrix Contractions

This section deconstructs the precise programmatic mechanics implemented to compute the curl-free field maps over large-scale dense mesh grids, matching the operations defined in our computational pipeline.

13.7.1 Block Interleave Index Gymnastics

The numerical assembly of a matrix-valued kernel demands structured populating of a $3N_1 \times 3N_2$ arrays space. Let $\mathbf{X}_1$ and $\mathbf{X}_2$ represent two coordinate matrices of length N_1 and N_2 respectively. The distance-displacement tensor is computed pairwise:

$$r_{i,j,d} = [\mathbf{X}_1]_{i,d} - [\mathbf{X}_2]_{j,d} \quad \forall\ d \in \{0, 1, 2\} \tag{130}$$

To preserve structural coherence across individual directional axes, our numerical solver organises the matrix using index-stride slicing. For a chosen row dimension d_1 and column dimension d_2, the component block matrix is mapped to the global matrix $\mathbf{K}$ using a stride factor of 3:

$$\mathbf{K}[d_1 :: 3,\ d_2 :: 3] = \sigma_f^2 \exp\left(-\frac{\|\mathbf{r}_{i,j}\|^2}{2\ell^2}\right) \odot \left[\delta_{d_1,d_2} - \frac{r_{i,j,d_1} r_{i,j,d_2}}{\ell^2}\right] \tag{131}$$

This index strategy splits the final array such that the first three rows represent the $[B_N, B_E, B_D]$ interactions of the first survey node, followed sequentially by subsequent spatial coordinates.

13.7.2 Spatial Voxel Thinning vs. High-Density Evaluation Grid

To construct a computationally tractable and memory-safe pipeline, our architecture enforces a strict decoupling between the training spatial resolution and the final evaluation grid resolution.

1. Phase 1: Voxel Thinning (1.0m Resolution). During data ingestion, the raw high-frequency telemetry data is spatially thinned using a 1.0-meter voxel filter. This clusters and averages redundant sensor readings, reducing the dataset down to N unique spatial cells. This aggressive downsampling is physically justified (the ambient crustal field does not vary significantly at sub-meter resolutions) and mathematically

critical: it prevents matrix singularity caused by co-located coordinates and strictly bounds the $\mathcal{O}(N^3)$ computational complexity of inverting the global training matrix $\mathbf{K}_y$.

2. Phase 3: Tensor Baking (20cm Resolution). Conversely, during the validation map baking phase, we evaluate N_{grid} spatial coordinates across a much denser 20-centimeter resolution meshgrid. This high-fidelity interpolation ensures smooth, continuous gradients for the downstream Unscented Kalman Filter lookup tensor ($\mathcal{T}_{\text{map}}$).

However, querying this dense meshgrid poses a critical bottleneck. The full predictive covariance equation yields a matrix of dimensions $3N_{\text{grid}} \times 3N_{\text{grid}}$. Because our multi-sensor Kalman filter lookup update (Eq. 45) only requests the point-wise marginal variance $\sigma_F^2(\mathbf{x}_*)$ at the specific query point, we completely disregard the cross-covariance between separate grid points.

Thus, we only require the absolute diagonal entries of the global system. Let $\mathbf{K}_* \in \mathbb{R}^{3N_{\text{grid}} \times 3N}$ match our cross-covariance block matrix. We avoid a direct matrix inversion by utilising an unpivoted symmetric linear solver to compute an intermediate matrix array $\mathbf{M} \in \mathbb{R}^{3N \times 3N_{\text{grid}}}$:

$$\mathbf{K}_y \mathbf{M} = \mathbf{K}_*^T \implies \mathbf{M} = \mathbf{K}_y^{-1} \mathbf{K}_*^T \tag{132}$$

The target variance vector $\mathbf{v}_{\text{flat}} \in \mathbb{R}^{3N_{\text{grid}} \times 1}$ represents the isolated diagonal elements of the latent matrix multiplication:

$$\mathbf{v}_{\text{flat}} = \text{diag}\left(\sigma_f^2 \mathbf{I} - \mathbf{K}_* \mathbf{M}\right) \tag{133}$$

Invoking the linear algebra identity for the diagonal entries of a matrix product ($\text{diag}(\mathbf{AB}) = \sum_{\text{cols}} \mathbf{A} \odot \mathbf{B}^T$), we can evaluate this entire reduction field without ever materialising the giant $3N_{\text{grid}} \times 3N_{\text{grid}}$ array in memory:

$$[\mathbf{v}_{\text{flat}}]_i = \sigma_f^2 - \sum_{j=1}^{3N} [\mathbf{K}_*]_{i,j} \cdot [\mathbf{M}]_{j,i} \tag{134}$$

By computing this reduction via a row-wise sum contraction, the memory allocation bounds drop from a quadratic footprint $\mathcal{O}(N_{\text{grid}}^2)$ to a completely safe, linear complexity profile $\mathcal{O}(N_{\text{grid}} \cdot N)$. This optimisation enables our dense 20-centimeter evaluation grids to be processed efficiently in batches without causing system RAM exhaustion. The final output is seamlessly structured into 3D spatial tensors tracking the predictive means $(\mu_{bx}, \mu_{by}, \mu_{bz}, \mu_{bf})$ and their respective epistemic variances $(\sigma_{bx}^2, \sigma_{by}^2, \sigma_{bz}^2, \sigma_{bf}^2)$ for real-time navigation lookup.

Summary In summary, what we have done over this entire section is to explain and derive how we model magnetic fields. The core philosophy is to assume that everything is a gaussian, while accounting for physically realisable magnetic fields under Maxwellian constraints. We assume that sensor noise is gaussian, we assume that the correlations between spatial points are also gaussian and we also assume that the rate at which the B field readings fall off are also gaussian, all while taking into account the data collected in $\mathcal{D}$. From this, we can query any arbitrary point we want to produce not only the predicted average value of the B field (and its components) for any point in space, but also the uncertainty (variance) of all of the B field's components of the queried location.

References

[1] S. Scholes, D. Hunter, C. Dyer, M. Mrozowski, A. McWilliam, P. Utting, D. Burt, P. F. Griffin, J. McGilligan, E. Riis, and S. J. Ingleby, "Field validation of GNSS-independent positioning enhancement using a wearable ultra-stable quantum magnetometer," *arXiv preprint arXiv:2606.24612*, 2026.

[2] X. Wang, W. Li, B. Moran, B. C. Gibson, L. T. Hall, D. Simpson, A. N. Kealy, and A. D. Greentree, "Quantum diamond magnetometry for navigation in GNSS denied environments," *arXiv preprint arXiv:2302.06187*, 2023.

[3] F. Alhomayani and M. H. Mahoor, "Outfin, a multi-device and multi-modal dataset for outdoor localization based on the fingerprinting approach," *Scientific Data*, vol. 8, no. 1, p. 66, 2021.

[4] C. Huang, G. Hendeby, H. Fourati, C. Prieur, and I. Skog, "MAINS: A magnetic field aided inertial navigation system for indoor positioning," *arXiv preprint arXiv:2312.02599*, 2023.

[5] H. Rosé, K. Klipp, T. Koubek, B. Schäufele, and I. Radusch, "Magnetic indoor localization through CNN regression and rotation invariance," *arXiv preprint arXiv:2604.22896*, 2026.

[6] National Geospatial-Intelligence Agency, "NGA, NOAA, BGS release world magnetic model 2025." `https://www.nga.mil/news/NGA_NOAA_BGS_Publish_World_Magnetic_Model_2025.html`, Dec. 2024. Accessed: 2026-07-27.

[7] Q-CTRL, "Time names q-ctrl ironstone opal one of the best inventions of 2025," 2025.

[8] Q-CTRL, "Q-ctrl overcomes gps-denial with quantum sensing, achieves quantum advantage," 2025.

[9] K. Choi, "Sensor logger: One-page unit reference," 2026.

[10] Donglin Si, "A study on the treatment of temperature drift in fluxgate magnetometers." `https://ieeexplore.ieee.org/document/10878699`, 2025. Accessed: 2026-07-28.

[11] R. Labbe, *Kalman and Bayesian Filters in Python*. GitHub, 2014.

[12] H. Bijl, "Gaussian process regression techniques - the source code corresponding to the ph.d. thesis.," 2017.

[13] J. Cheng, *Ground User Localization in UAV-based Wireless Networks: From Channel Characterization to System Design*. Ph.d. dissertation, Université Libre de Bruxelles, 2022.